

# NUMB11: Numerical Linear Algebra

Arpita Bhattacharya

November 13, 2024

## 1. Task:

Let  $A$  be a square matrix,  $\rho(A)$  be the spectral radius of  $A$  and let  $\|\cdot\|$  any operator norm. Show that  $\rho(A) \leq \|A\|$ .

## Solution:

$A$  is a square matrix of dimensions  $n \times n$ .  $\rho(A)$  is the spectral radius of  $A$ . We need to show that  $\rho(A) \leq \|A\|$ .

We know that

$$\rho(A) = \max_{1 \leq i \leq n} |\lambda_i(A)| \quad (1)$$

Or, the largest eigenvalue of matrix  $A$ .

If  $\lambda$  is the largest eigenvalue of matrix  $A$ , then there is a vector  $x$  such that

$$Ax = \lambda x \quad (2)$$

This is the eigenvector. Taking norm on both sides,

$$\|Ax\| = \|\lambda x\| \quad (3)$$

Since  $\lambda$  is a scalar,

$$\begin{aligned} \|Ax\| &= |\lambda| \|x\| \\ |\lambda| &= \frac{\|Ax\|}{\|x\|} \\ &\leq \sup_{x \neq 0} \frac{\|Ax\|}{\|x\|} \\ &= \|A\| \end{aligned} \quad (4)$$

So now we have got that  $|\lambda| \leq \|A\|$  where  $\rho(A) = \lambda$ .

Hence,

$$\rho(A) \leq \|A\| \quad (5)$$

**2. Task:**

Use the statement in a previous task to show that

$$\|A\| < 1 \implies \lim_{n \rightarrow \infty} A^n = 0$$

where  $A$  is a square matrix,  $0$  the zero matrix of corresponding dimensions and  $\|\cdot\|$  an operator norm.

**Solution:**

$A$  is a square matrix and  $0$  is the zero matrix of corresponding dimensions. We need to show that  $\|A\| < 1 \implies \lim_{n \rightarrow \infty} A^n = 0$ .

We know from a previous task that  $\rho(A) \leq \|A\|$ . Now if  $\|A\| < 1$ , then  $\rho(A) \leq \|A\| < 1$ . Taking power of  $n$  on both sides,

$$\begin{aligned} (\rho(A))^n &\leq \|A\|^n \\ (\rho(A))^n &\leq \|A\|^n < 1 \end{aligned} \tag{6}$$

If  $\lambda$  is the largest eigenvalue of matrix  $A$ , then,

$$\begin{aligned} (|\lambda|)^n &< 1 \\ |\lambda| &< 1 \end{aligned} \tag{7}$$

This implies that  $\lim_{n \rightarrow \infty} \lambda^n = 0$ .

If  $\lambda$  is the largest eigenvalue of matrix  $A$ , then there is a vector  $x$  such that

$$\begin{aligned} Ax &= \lambda x \\ A^k x &= \lambda^k x \quad \text{for } k = 0, 1, 2, \dots \end{aligned} \tag{8}$$

So,

$$\begin{aligned} \lim_{n \rightarrow \infty} \lambda^n &= 0 \implies x \cdot \lim_{n \rightarrow \infty} \lambda^n = 0 \\ &\implies \lim_{n \rightarrow \infty} (\lambda^n x) = 0 \\ &\implies \lim_{n \rightarrow \infty} (A^n x) = 0 \\ &\implies \left( \lim_{n \rightarrow \infty} A^n \right) x = 0 \\ &\implies \lim_{n \rightarrow \infty} A^n = 0 \end{aligned} \tag{9}$$

Thus,

$$\|A\| < 1 \implies \lim_{n \rightarrow \infty} A^n = 0 \tag{10}$$

**3. Task:**

This task is related to the equivalence of norms theorem. Let  $A$  be an  $n \times n$  matrix and  $x$  an  $n$  vector. Show,

**3.1.**  $\|x\|_\infty \leq \|x\|_2$

We know that  $\|x\|_\infty = \max_{1 \leq i \leq n} |x_i|$ .

Let  $1 \leq j \leq n$  such that  $\max_{1 \leq i \leq n} |x_i| = |x_j|$ .

So,

$$\|x\|_\infty = |x_j| \quad (11)$$

We know that

$$\|x\|_2 = \left( \sum_{i=1}^n |x_i|^2 \right)^{\frac{1}{2}} \quad (12)$$

For  $1 \leq j \leq n$ ,

$$\begin{aligned} \left( \sum_{i=1}^n |x_i|^2 \right)^{\frac{1}{2}} &= \left( |x_j|^2 + \sum_{i=1, i \neq j}^n |x_i|^2 \right)^{\frac{1}{2}} \\ &\geq (|x_j|^2)^{\frac{1}{2}} \\ &= |x_j| = \|x\|_\infty \end{aligned} \quad (13)$$

From 11 and 13, we can conclude that,

$$\|x\|_\infty \leq \|x\|_2 \quad (14)$$

**3.2.**  $\|x\|_2 \leq \sqrt{n} \|x\|_\infty$

We know that,

$$\|x\|_2 = \left( \sum_{i=1}^n |x_i|^2 \right)^{\frac{1}{2}} \quad (15)$$

We can rewrite this as,

$$\begin{aligned} \|x\|_2 &= \left( \sum_{i=1}^n |x_i|^2 \right)^{\frac{1}{2}} \\ &= (|x_1|^2 + |x_2|^2 + \dots + |x_n|^2)^{\frac{1}{2}} \\ &\leq \left( \max_{1 \leq i \leq n} |x_i|^2 + \max_{1 \leq i \leq n} |x_i|^2 + \dots + \max_{1 \leq i \leq n} |x_i|^2 \right)^{\frac{1}{2}} \\ &= \left( n \max_{1 \leq i \leq n} |x_i|^2 \right)^{\frac{1}{2}} \\ &= \sqrt{n} \max_{1 \leq i \leq n} |x_i|^2 \\ &= \sqrt{n} \|x\|_\infty \end{aligned} \quad (16)$$

Thus,

$$\|x\|_2 \leq \sqrt{n} \|x\|_\infty \quad (17)$$

**3.3.**  $\|A\|_\infty \leq \sqrt{n}\|A\|_2$

From 3.1 we know that,

$$\begin{aligned} \|x\|_\infty &\leq \|x\|_2 \\ \implies \|Ax\|_\infty &\leq \|Ax\|_2 \end{aligned} \quad (18)$$

And, from 3.2 we know that,

$$\begin{aligned} \|x\|_2 &\leq \sqrt{n}\|x\|_\infty \\ \implies \frac{1}{\|x\|_2} &\geq \frac{1}{\sqrt{n}\|x\|_\infty}, \quad (x \neq 0) \end{aligned} \quad (19)$$

Using 18 and 19,

$$\begin{aligned} \frac{\|Ax\|_\infty}{\sqrt{n}\|x\|_\infty} &\leq \frac{\|Ax\|_2}{\|x\|_2}, \quad (x \neq 0) \\ \implies \frac{\|Ax\|_\infty}{\|x\|_\infty} &\leq \frac{\sqrt{n}\|Ax\|_2}{\|x\|_2}, \quad (x \neq 0) \end{aligned} \quad (20)$$

Taking supremum on both sides we finally get that,

$$\|A\|_\infty \leq \sqrt{n}\|A\|_2 \quad (21)$$

**3.4.**  $\|A\|_2 \leq \sqrt{n}\|A\|_\infty$

From 3.1 we know that,

$$\begin{aligned} \|x\|_\infty &\leq \|x\|_2 \\ \implies \frac{1}{\|x\|_\infty} &\geq \frac{1}{\|x\|_2}, \quad (x \neq 0) \end{aligned} \quad (22)$$

And, from 3.2 we know that,

$$\begin{aligned} \|x\|_2 &\leq \sqrt{n}\|x\|_\infty \\ \implies \|Ax\|_2 &\leq \sqrt{n}\|Ax\|_\infty \end{aligned} \quad (23)$$

Using 22 and 23,

$$\frac{\sqrt{n}\|Ax\|_\infty}{\|x\|_\infty} \geq \frac{\|Ax\|_2}{\|x\|_2}, \quad (x \neq 0) \quad (24)$$

Taking supremum on both sides we finally get that,

$$\begin{aligned} \sqrt{n}\|A\|_\infty &\geq \|A\|_2 \\ \text{or, } \|A\|_2 &\leq \sqrt{n}\|A\|_\infty \end{aligned} \quad (25)$$

4. **Task:**

Consider a row matrix  $A$  and a vector  $a$  with the same entries as  $A$ . Show that  $\|A\|_2 = \|a\|_2$ , where on the left side of the equation the matrix norm is used and on the right hand side the vector norm. What about the 1- and 1-norm? Can one prove a similar theorem or can you find a counter example?

**Solution:**

A row matrix  $A$  and a vector  $a$  that have the same entries. That is,

$$A = [a_1 \quad a_2 \quad \cdots \quad a_n] \quad \text{and} \quad a = \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_n \end{bmatrix} \quad (26)$$

We know that,

$$\begin{aligned} \|A\|_2 &= \sqrt{\rho(A^T A)} \\ \text{and} \quad \|a\|_2 &= \sqrt{a_1^2 + a_2^2 + \cdots + a_n^2} \end{aligned} \quad (27)$$

Next we will try to find what  $\rho(A^T A)$  is. First,

$$A^T = \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_n \end{bmatrix} \quad (28)$$

Hence,

$$A^T A = \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_n \end{bmatrix} [a_1 \quad a_2 \quad \cdots \quad a_n] = \begin{bmatrix} a_1^2 & a_1 a_2 & \cdots & a_1 a_n \\ a_2 a_1 & a_2^2 & \cdots & a_2 a_n \\ \vdots & \vdots & \ddots & \vdots \\ a_n a_1 & a_n a_2 & \cdots & a_n^2 \end{bmatrix} \quad (29)$$

We get a  $n \times n$  matrix.  $A^T A$  is symmetric. It's eigenvalues are non-negative. In fact, it's only non-zero eigenvalue is,  $a_1^2 + a_2^2 + \cdots + a_n^2$ . Thus,

$$\rho(A^T A) = a_1^2 + a_2^2 + \cdots + a_n^2 \quad (30)$$

Therefore,

$$\begin{aligned} \|A\|_2 &= \sqrt{\rho(A^T A)} \\ &= \sqrt{a_1^2 + a_2^2 + \cdots + a_n^2} \\ &= \|a\|_2 \end{aligned} \quad (31)$$

We have hence proved that  $\|A\|_2 = \|a\|_2$ .

Next, to find something similar for 1-norm and  $\infty$ -norm.

We know that following,

$$\begin{aligned}
\|a\|_1 &= |a_1| + |a_2| + \cdots + |a_n| \\
\|a\|_\infty &= \max_{1 \leq i \leq n} |a_i| \\
\|A\|_1 &= \max_{1 \leq j \leq n} \sum_{1 \leq i \leq n} |a_{ij}| \\
\|A\|_\infty &= \max_{1 \leq i \leq n} \sum_{1 \leq j \leq n} |a_{ij}|
\end{aligned} \tag{32}$$

$\|A\|_1$  is basically finding the sum of the absolute values of each column and then taking the biggest value.

Or, for the given row matrix A,

$$\begin{aligned}
\|A\|_1 &= \max_{1 \leq i \leq n} |a_i| \\
&= \|a\|_\infty
\end{aligned} \tag{33}$$

And,  $\|A\|_\infty$  is basically finding the sum of the absolute values of each row and then taking the biggest value.

Or, for the given row matrix A,

$$\begin{aligned}
\|A\|_\infty &= \sum_{1 \leq i \leq n} |a_i| \\
&= \|a\|_1
\end{aligned} \tag{34}$$

Thus, we have found that,

$$\begin{aligned}
&\|A\|_1 = \|a\|_\infty \\
\text{and } \|A\|_\infty &= \|a\|_1
\end{aligned} \tag{35}$$

## 5. Task:

Show for the Frobenius norm, that  $\|A\|_F = \sqrt{\text{trace}(A^T A)}$  holds for any matrix A. Use this fact to show that  $\|QA\|_F = \|A\|_F$  for any matrix A and any orthogonal matrix Q with suitable dimensions.

## Solution:

For any matrix A, the Frobenius norm is,

$$\|A\|_F = \sqrt{\sum_{i=1}^m \sum_{j=1}^n |a_{ij}|^2} \tag{36}$$

where  $a_{ij} \in A$  and  $A \in \mathbb{R}^{m \times n}$

When multiplying matrix A by it's transpose  $A^T$ ,  $A^T A$  is a square matrix. The element in the  $i^{\text{th}}$  row and  $i^{\text{th}}$  column of  $A^T A$ , or the diagonal elements is given by,

$$(A^T A)_{ii} = \sum_{k=1}^n a_{ki} \cdot a_{ki} = \sum_{k=1}^n a_{ki}^2 \tag{37}$$

$\text{Trace}(A^T A)$  is the sum of these diagonal elements.

$$\text{trace}(A^T A) = \sum_{i=1}^n (A^T A)_{ii} = \sum_{i=1}^m \sum_{k=1}^n a_{ki}^2 \quad (38)$$

From 36 and 38, we can conclude that,

$$\|A\|_F = \sqrt{\text{trace}(A^T A)} \quad (39)$$

Next, we have that  $Q$  is an orthogonal matrix with suitable dimensions. It has a property  $Q^T Q = I$  where  $I$  is the identity matrix. Now, using the previous result,

$$\begin{aligned} \|QA\|_F &= \sqrt{\text{trace}((QA)^T(QA))} \\ &= \sqrt{\text{trace}((A^T Q^T)(QA))} \\ &= \sqrt{\text{trace}(A^T Q^T Q A)} \quad (Q^T Q = I) \\ &= \sqrt{\text{trace}(A^T A)} \\ &= \|A\|_F \end{aligned} \quad (40)$$

Thus,  $\|QA\|_F = \|A\|_F$

#### 6. Task:

Let  $u, v$  be two  $n$  vectors. Show that their outer product,  $uv^T$ , is an  $n \times n$  matrix of rank 1. Compute its 2-norm.

#### Solution:

$\mathbf{u}, \mathbf{v}$  are 2 vectors of  $n$  dimension. So,

$$\mathbf{uv}^T = \begin{bmatrix} u_1 \\ u_2 \\ \vdots \\ u_n \end{bmatrix} \begin{bmatrix} v_1 & v_2 & \cdots & v_n \end{bmatrix} \quad (41)$$

We know that rank of a matrix  $A$  is the dimension of the vector space spanned (or generated) by its columns (or rows). That is, it's the maximal number of linearly independent columns (or rows). Now,

$$\mathbf{uv}^T = \begin{bmatrix} u_1 \\ u_2 \\ \vdots \\ u_n \end{bmatrix} \begin{bmatrix} v_1 & v_2 & \cdots & v_n \end{bmatrix} = \begin{bmatrix} u_1 v_1 & u_1 v_2 & \cdots & u_1 v_n \\ u_2 v_1 & u_2 v_2 & \cdots & u_2 v_n \\ \vdots & \vdots & \ddots & \vdots \\ u_n v_1 & u_n v_2 & \cdots & u_n v_n \end{bmatrix} \quad (42)$$

Columns of the outer product are all proportional to the first column. Therefore, they are all linearly dependent on that one column.

Or we can say that, all columns (or rows) of a matrix are scalar multiples of each other or, that each row is a multiple of  $\mathbf{v}$  and each column is a multiple of  $\mathbf{u}$ .

This means that the matrix can be described by a single row or column. Which means that only one column (or row) is needed to describe the entire matrix.

Thus, we can say that they are linearly dependent. And there is only one column (or row) that is linearly independent, one independent vector. (The rest being scalar multiples of this).

Hence, the outer product  $\mathbf{u}\mathbf{v}^T$  is a  $n \times n$  matrix of rank 1.

Next, computing the 2-norm of the outer product,  $\mathbf{u}\mathbf{v}^T = A$

$$\begin{aligned}
\|A\|_2 &= \sqrt{\rho((A)^T(A))} \\
&= \sqrt{\rho((\mathbf{u}\mathbf{v}^T)^T(\mathbf{u}\mathbf{v}^T))} \\
&= \sqrt{\rho(\mathbf{v}\mathbf{u}^T\mathbf{u}\mathbf{v}^T)} \\
&= \sqrt{\mathbf{u}^T\mathbf{u}}\sqrt{\rho(\mathbf{v}\mathbf{v}^T)} \quad \left( \rho(\mathbf{u}^T\mathbf{u}) = \mathbf{u}^T\mathbf{u} = \sum_{i=1}^n u_i^2, \quad \text{a scalar value} \right)
\end{aligned} \tag{43}$$

$\mathbf{v}\mathbf{v}^T$  is a square matrix with rank 1. It has eigenvalues 0 and the trace  $= \sum_{i=1}^n v_i^2$ . Thus, the largest eigenvalue is  $\sum_{i=1}^n v_i^2$ .

$$\begin{aligned}
&= \sqrt{\left(\sum_{i=1}^n u_i^2\right)\left(\sum_{i=1}^n v_i^2\right)} \\
&= \|u\|_2 \|v\|_2
\end{aligned} \tag{44}$$

Thus, the 2-norm of the outer product is  $\|u\|_2 \|v\|_2$ .

#### 7. Task:

Let  $uv^T$  be the outer product of two vectors. Give its singular value decomposition in terms of these two vectors. Make also a numeric example and verify your statement in Python/ MATLAB (for Python you have to import `scipy.linalg`).

#### Solution:

$\mathbf{u}\mathbf{v}^T$  is the outer product of two vectors. We need to find it's singular value decomposition (SVD) in terms of these two vectors.

SVD of a matrix  $A = U\Sigma V^T$  where,

$U = m \times m$  orthogonal matrix

$\Sigma = m \times n$  diagonal matrix with non-negative real values in the diagonal (singular values)



$V^T = n \times n$  orthogonal matrix  $V$ 's transpose

There are vectors  $\mathbf{u}_1, \dots, \mathbf{u}_m$  such that,

$$U = [\mathbf{u}_1 \quad \mathbf{u}_2 \quad \cdots \quad \mathbf{u}_m] \quad (45)$$

And there are vectors  $\mathbf{v}_1, \dots, \mathbf{v}_n$  such that,

$$V^T = \begin{bmatrix} \mathbf{v}_1^T \\ \mathbf{v}_2^T \\ \vdots \\ \mathbf{v}_n^T \end{bmatrix} \quad (46)$$

$$\text{Let } \Sigma = \begin{bmatrix} \sigma_1 & \cdots & 0 & \\ \vdots & \ddots & \vdots & O \\ 0 & \cdots & \sigma_r & \\ & O & & O \end{bmatrix}$$

Then,

$$\begin{aligned} A &= U \Sigma V^T \\ &= [\mathbf{u}_1 \quad \mathbf{u}_2 \quad \cdots \quad \mathbf{u}_m] \begin{bmatrix} \sigma_1 & \cdots & 0 & \\ \vdots & \ddots & \vdots & O \\ 0 & \cdots & \sigma_r & \\ & O & & O \end{bmatrix} \begin{bmatrix} \mathbf{v}_1^T \\ \mathbf{v}_2^T \\ \vdots \\ \mathbf{v}_n^T \end{bmatrix} \end{aligned} \quad (47)$$

$$\begin{aligned} &= [\mathbf{u}_1 \quad \cdots \quad \mathbf{u}_r \quad \mathbf{u}_{r+1} \quad \cdots \quad \mathbf{u}_m] \begin{bmatrix} \sigma_1 & \cdots & 0 & \\ \vdots & \ddots & \vdots & O \\ 0 & \cdots & \sigma_r & \\ & O & & O \end{bmatrix} \begin{bmatrix} \mathbf{v}_1^T \\ \vdots \\ \mathbf{v}_r^T \\ \mathbf{v}_{r+1}^T \\ \vdots \\ \mathbf{v}_n^T \end{bmatrix} \\ &= [\mathbf{u}_1 \quad \cdots \quad \mathbf{u}_r] \begin{bmatrix} \sigma_1 & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & \sigma_r \end{bmatrix} \begin{bmatrix} \mathbf{v}_1^T \\ \vdots \\ \mathbf{v}_r^T \end{bmatrix} + [\mathbf{u}_{r+1} \quad \cdots \quad \mathbf{u}_m] [O] \begin{bmatrix} \mathbf{v}_{r+1}^T \\ \vdots \\ \mathbf{v}_n^T \end{bmatrix} \\ &= [\sigma_1 \mathbf{u}_1 \quad \cdots \quad \sigma_r \mathbf{u}_r] \begin{bmatrix} \mathbf{v}_1^T \\ \vdots \\ \mathbf{v}_r^T \end{bmatrix} \\ &= \sigma_1 \mathbf{u}_1 \mathbf{v}_1^T + \dots + \sigma_r \mathbf{u}_r \mathbf{v}_r^T \\ &= \sum_{i=1}^r \sigma_i \mathbf{u}_i \mathbf{v}_i^T \end{aligned} \quad (48)$$

Thus,

$$A = U\Sigma V^T = \sum_{i=1}^r \sigma_i \mathbf{u}_i \mathbf{v}_i^T \quad (49)$$

where  $r$  is the rank of the matrix  $A$  and  $\mathbf{u}_i \mathbf{v}_i^T$  are the outer products.

Next, let's provide a numeric example: Let  $A = \begin{bmatrix} 3 & 1 & 1 \\ -1 & 3 & 1 \end{bmatrix}$

Let's begin with finding  $U$ .

$$A^T = \begin{bmatrix} 3 & -1 \\ 1 & 3 \\ 1 & 1 \end{bmatrix}$$

$$AA^T = \begin{bmatrix} 3 & 1 & 1 \\ -1 & 3 & 1 \end{bmatrix} \begin{bmatrix} 3 & -1 \\ 1 & 3 \\ 1 & 1 \end{bmatrix} = \begin{bmatrix} 11 & 1 \\ 1 & 11 \end{bmatrix}$$

Next, we have to find the eigenvalues and corresponding eigenvectors of  $AA^T$ . We know that eigenvectors are defined by the equation  $\vec{A}v = \lambda v$ , and applying this to  $AA^T$  gives us

$$\begin{bmatrix} 11 & 1 \\ 1 & 11 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \lambda \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \quad (50)$$

Rewriting it as,

$$\begin{aligned} 11x_1 + x_2 &= \lambda x_1 \\ x_1 + 11x_2 &= \lambda x_2 \end{aligned} \quad (51)$$

$$\begin{aligned} \text{Or } (11 - \lambda)x_1 + x_2 &= 0 \\ x_1 + (11 - \lambda)x_2 &= 0 \end{aligned}$$

Solve for  $\lambda$  by taking the determinant equal to zero.

$$\begin{aligned} \begin{vmatrix} (11 - \lambda) & 1 \\ 1 & (11 - \lambda) \end{vmatrix} &= 0 \\ (11 - \lambda)(11 - \lambda) - 1 &= 0 \\ (\lambda - 10)(\lambda - 12) &= 0 \\ \lambda = 10, \lambda = 12 \end{aligned} \quad (52)$$

This gives us two eigenvalues  $\lambda = 10, 12$ . Plugging  $\lambda$  back into the original equations gives us our eigenvectors. For  $\lambda = 10$ , we get eigenvector  $\begin{bmatrix} 1 & -1 \end{bmatrix}$ . And, for  $\lambda = 12$ , we get eigenvector  $\begin{bmatrix} 1 & 1 \end{bmatrix}$

We can write these eigenvectors as a matrix from the largest to smallest eigenvalue's eigenvector from the first column on the left towards the right.

$$\begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \quad (53)$$

We can convert this matrix into an orthogonal matrix by applying Gram-Schmidt orthonormalization. First, we'll normalize  $\vec{v}_1$ .

$$\vec{u}_1 = \frac{\vec{v}_1}{|\vec{v}_1|} = \frac{[1, 1]}{\sqrt{1^2 + 1^2}} = \frac{[1, 1]}{\sqrt{2}} = \left[\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}}\right] \quad (54)$$

Compute

$$\begin{aligned} \vec{w}_2 &= \vec{v}_2 - \vec{u}_1 \cdot \vec{v}_2 * \vec{u}_1 \\ &= [1, -1] - \left[\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}}\right] \cdot [1, -1] * \left[\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}}\right] \\ &= [1, -1] - 0 * \left[\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}}\right] \\ &= [1, -1] - [0, 0] \\ &= [1, -1] \end{aligned} \quad (55)$$

and normalizing to give  $U$ .

$$\vec{u}_2 = \frac{\vec{w}_2}{|\vec{w}_2|} = \left[\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}}\right] \quad (56)$$

$$U = \begin{bmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \end{bmatrix} \quad (57)$$

Next, we will find out what  $V$  is.  $V$  is similarly calculated based on  $A^T A$ .

$$A^T A = \begin{bmatrix} 3 & -1 \\ 1 & 3 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 3 & 1 & 1 \\ -1 & 3 & 1 \end{bmatrix} = \begin{bmatrix} 10 & 0 & 2 \\ 0 & 10 & 4 \\ 2 & 4 & 2 \end{bmatrix} \quad (58)$$

In the same way we did for  $U$ , we find the eigenvalues of matrix  $A^T A$  as  $\lambda_1 = 0$ ,  $\lambda_2 = 10$  and  $\lambda_3 = 12$  using  $(A^T A) - \lambda I = 0$ . And we find the corresponding eigenvectors as  $\vec{v}_1 = [1, 2, -5]$ ,  $\vec{v}_2 = [2, -1, 0]$  and  $\vec{v}_3 = [1, 2, 1]$  respectively using  $(A^T A)\vec{v} = \lambda\vec{v}$ .

We will order  $\vec{v}_1$ ,  $\vec{v}_2$  and  $\vec{v}_3$  as column vectors in a matrix according to size of the eigenvalue to get,

$$\begin{bmatrix} 1 & 2 & 1 \\ 2 & -1 & 2 \\ 1 & 0 & -5 \end{bmatrix} \quad (59)$$

We can convert this matrix into an orthogonal matrix by applying Gram-Schmidt orthonormalization.

$$\begin{aligned}
\vec{u}_1 &= \frac{\vec{v}_1}{|\vec{v}_1|} = \left[ \frac{1}{\sqrt{6}}, \frac{2}{\sqrt{6}}, \frac{1}{\sqrt{6}} \right] \\
\vec{w}_2 &= \vec{v}_2 - \vec{u}_1 \cdot \vec{v}_2 * \vec{u}_1 = [2, -1, 0] \\
\vec{u}_2 &= \frac{\vec{w}_2}{|\vec{w}_2|} = \left[ \frac{2}{\sqrt{5}}, \frac{-1}{\sqrt{5}}, 0 \right] \\
\vec{w}_3 &= \vec{v}_3 - \vec{u}_1 \cdot \vec{v}_3 * \vec{u}_1 - \vec{u}_2 \cdot \vec{v}_3 * \vec{u}_2 = \left[ \frac{-2}{3}, \frac{-4}{3}, \frac{10}{3} \right] \\
\vec{u}_3 &= \frac{\vec{w}_3}{|\vec{w}_3|} = \left[ \frac{1}{\sqrt{30}}, \frac{2}{\sqrt{30}}, \frac{-5}{\sqrt{30}} \right]
\end{aligned} \tag{60}$$

This would give,

$$V = \begin{bmatrix} \frac{1}{\sqrt{6}} & \frac{2}{\sqrt{5}} & \frac{1}{\sqrt{30}} \\ \frac{2}{\sqrt{6}} & \frac{-1}{\sqrt{5}} & \frac{2}{\sqrt{30}} \\ \frac{1}{\sqrt{6}} & 0 & \frac{-5}{\sqrt{30}} \end{bmatrix} \tag{61}$$

Thus,

$$V^T = \begin{bmatrix} \frac{1}{\sqrt{6}} & \frac{2}{\sqrt{6}} & \frac{1}{\sqrt{6}} \\ \frac{2}{\sqrt{5}} & \frac{-1}{\sqrt{5}} & 0 \\ \frac{1}{\sqrt{30}} & \frac{2}{\sqrt{30}} & \frac{-5}{\sqrt{30}} \end{bmatrix} \tag{62}$$

For  $\Sigma$ , we take square roots of the non-zero eigenvalues and write them as the diagonals putting the largest in the first column, next largest in the next column and so on until the smallest. The diagonal entries are the singular values of  $A$ . Hence,

$$\Sigma = \begin{bmatrix} \sqrt{12} & 0 & 0 \\ 0 & \sqrt{10} & 0 \end{bmatrix} \tag{63}$$

Thus, we finally have,

$$A = U\Sigma V^T = \begin{bmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & \frac{-1}{\sqrt{2}} \end{bmatrix} \begin{bmatrix} \sqrt{12} & 0 & 0 \\ 0 & \sqrt{10} & 0 \end{bmatrix} \begin{bmatrix} \frac{1}{\sqrt{6}} & \frac{2}{\sqrt{5}} & \frac{1}{\sqrt{30}} \\ \frac{2}{\sqrt{6}} & \frac{-1}{\sqrt{5}} & \frac{2}{\sqrt{30}} \\ \frac{1}{\sqrt{6}} & 0 & \frac{-5}{\sqrt{30}} \end{bmatrix} \tag{64}$$

Verifying we get,

$$A = U\Sigma V^T = \begin{bmatrix} \frac{\sqrt{12}}{\sqrt{2}} & \frac{\sqrt{10}}{\sqrt{2}} & 0 \\ \frac{\sqrt{12}}{\sqrt{2}} & \frac{-\sqrt{10}}{\sqrt{2}} & 0 \end{bmatrix} \begin{bmatrix} \frac{1}{\sqrt{6}} & \frac{2}{\sqrt{5}} & \frac{1}{\sqrt{30}} \\ \frac{2}{\sqrt{6}} & \frac{-1}{\sqrt{5}} & \frac{2}{\sqrt{30}} \\ \frac{1}{\sqrt{6}} & 0 & \frac{-5}{\sqrt{30}} \end{bmatrix} = \begin{bmatrix} 3 & 1 & 1 \\ -1 & 3 & 1 \end{bmatrix} \tag{65}$$

This gives the same solution as  $\sum_{i=1}^r \sigma_i \mathbf{u}_i \mathbf{v}_i^T = A$ .

We can verify 64 and 65 using Python programming.

**Python Code:**

```

import numpy as np
import scipy.linalg as sl

m = 2; n = 3

# Define the matrix A
A = np.array([[3, 1, 1], [-1, 3, 1]])

# Perform Singular Value Decomposition using scipy.linalg
U, s, V = sl.svd(A)
Sigma = sl.diagsvd(s,m,n)

# U is the left singular vectors
print("U:", U)

# s is the singular values.
# Sigma is the diagonal matrix with the singular values
print("Sigma:", Sigma)

# Vt is the transpose of the right singular vectors
print("Vt:", V)

```

Output:

```

U: [[-0.70710678 -0.70710678]
     [-0.70710678  0.70710678]]
Sigma: [[3.46410162 0.          0.          ]
         [0.          3.16227766 0.          ]
         [0.          0.          0.          ]]
Vt: [[-4.08248290e-01 -8.16496581e-01 -4.08248290e-01]
      [-8.94427191e-01  4.47213595e-01  5.26260748e-16]
      [-1.82574186e-01 -3.65148372e-01  9.12870929e-01]]

```

#### 8. Task:

In this Task, we are asked to apply the Gram-Schmidt orthogonalization to a given  $m \times n$  matrix  $A$  with  $m \geq n$ .

Using the concept of QR-factorization, we want to write matrix  $A = QR$  where  $Q$  is a  $m \times n$  matrix of orthonormal columns that span the successive subspaces spanned by the columns of  $A$ .  $R$  is a  $n \times n$  upper-triangular matrix.

$$[\mathbf{a}_1 \quad \mathbf{a}_2 \quad \cdots \quad \mathbf{a}_n] = [\mathbf{q}_1 \quad \mathbf{q}_2 \quad \cdots \quad \mathbf{q}_n] \begin{bmatrix} r_{11} & r_{12} & \cdots & r_{1n} \\ 0 & r_{22} & \cdots & r_{2n} \\ & & \ddots & \vdots \\ & 0 & & r_{nn} \end{bmatrix} \quad (66)$$

$a_1$  and  $q_1$  can be written as the linear combination of each other:

$$\begin{aligned}
a_1 &= r_{11}q_1, \\
a_2 &= r_{12}q_1 + r_{22}q_2, \\
a_3 &= r_{13}q_1 + r_{23}q_2 + r_{33}q_3, \\
&\vdots \\
a_n &= r_{1n}q_1 + r_{2n}q_2 + \cdots + r_{nn}q_n,
\end{aligned} \tag{67}$$

These equations is used to compute the reduced QR-factorizations. That is, given  $a_1, a_2, \dots$ , we can construct vectors  $q_1, q_2, \dots$  and entries  $r_{ij}$  by the Gram-Schmidt orthogonalization method.

In this method, at the  $j$ -th step, we will find a unit vector  $q_j$  that is orthogonal to  $q_1, \dots, q_{j-1}$ . In my python code, I have done this under the method `gramschmidt()`.

In this, I first run an outer loop where every column vector  $a_j$  is assigned to a vector  $v_j$ . So, this is the current vector that we will orthogonalize against the previous vectors. Then I run a second inner loop for this orthogonalization. This loop iterates over all previously computed orthonormal vectors  $q_i$  for  $i < j$ , to make  $v_j$  orthogonal to them. We will subtract the projection of  $v_j$  onto  $q_i$  to remove its component in that direction, making  $v_j$  orthogonal to  $q_i$ . Or,

$$v_j = a_j - (q_1^T a_j)q_1 - (q_2^T a_j)q_2 - \cdots - (q_{j-1}^T a_j)q_{j-1} \tag{68}$$

Now, if we rewrite the equations (67) as,

$$\begin{aligned}
q_1 &= \frac{a_1}{r_{11}}, \\
q_1 &= \frac{a_1 - r_{12}q_1}{r_{22}}, \\
q_1 &= \frac{a_1 - r_{13}q_1 - r_{23}q_2}{r_{33}}, \\
&\vdots \\
q_1 &= \frac{a_n - \sum_{i=1}^{n-1} r_{in}q_i}{r_{nn}},
\end{aligned} \tag{69}$$

Comparing this with equation (68), we have,

$$r_{ij} = q_i^T a_j \quad (i \neq j) \tag{70}$$

The coefficients  $r_{jj}$  in the denominators are chosen for normalization:

$$|r_{jj}| = \|a_j - \sum_{i=1}^{j-1} r_{ij}q_i\|_2 \tag{71}$$

We then finally normalize the vector  $v_j$  by dividing it by  $\|v_j\|_2$ . These vectors form the matrix  $Q$  and are the  $n$  orthogonal basis vectors of  $\text{range}(A)$ .

```

import numpy as np

class Orthogonalization:
    def __init__(self, A):

        self.A = np.array(A)
        self.m, self.n = self.A.shape

        if self.m < self.n:
            raise ValueError("The number of rows m is less than the number of
                               columns n.")

    def gramschmidt(self):

        Q = np.zeros((self.m, self.n)) # Matrix of the orthonormal vectors
        R = np.zeros((self.n, self.n)) # Upper triangular matrix

        for j in range(self.n):
            v_j = self.A[:, j] # Start with the j-th column of A

            # Subtract the projection onto the previous q_i vectors
            for i in range(j):
                q_i = Q[:, i]
                R[i, j] = np.dot(q_i, v_j) # r_ij = q_i^T * a_j
                v_j = v_j - R[i, j] * q_i # v_j = v_j - r_ij * q_i

            # Compute r_jj = ||v_j||_2 (2-norm of v_j)
            R[j, j] = np.linalg.norm(v_j, 2)

            # Normalize v_j to get q_j
            Q[:, j] = v_j / R[j, j]

        return Q, R

```

Add the following code to the above one as an example to find the orthogonal basis.

```

if __name__ == "__main__":
    A = [[1, 1, 0],
          [1, 0, 1],
          [0, 1, 1]]

    # Instantiate the class
    orth = Orthogonalization(A)

    # Perform Classical Gram-Schmidt orthogonalization
    Q, R = orth.gramschmidt()

    # Print the orthonormal basis vectors Q and the upper triangular matrix R
    print("Orthogonal basis vectors Q:")
    print(Q)
    print("\nUpper triangular matrix R:")
    print(R)

```

## 9. Task:

In this Task, we are asked to test the code in the previous task by choosing some random matrices. That is, to measuring the orthogonality of the matrix by a few different criteria. We are also required to test it with increasing dimensions of matrix  $A$ .

Let us first go through each criteria and write what outcomes we expect to occur:

- Is the 2-norm one?
  - For an orthonormal matrix, each column vector should have a 2-norm equal to 1.
- How big is the deviation of  $Q^T Q$  from the identity matrix? Measure this in 2-norm.
  - For an orthonormal matrix,  $Q^T Q$  should equal the identity matrix. If  $Q$  is perfectly orthonormal,  $Q^T Q - I$  should be close to zero. The 2-norm of this difference matrix gives us a measure of deviation.
- Compute the eigenvalues of  $Q^T Q$ . What do you expect they should be for an orthogonal matrix?
  - For an orthonormal matrix, all the eigenvalues of  $Q^T Q$  should be 1.
- Compute the determinant.
  - For an orthonormal matrix, the determinant should either be 1 or -1.

To measure the orthogonality based on the above criteria on some random matrices, we will use the code as written below. The output is also given there. Note that to find the determinant,  $Q$  needs to be a square matrix. Thus, I have chosen  $3 \times 3$  matrices.

Now, in the code, the method `checkorthogonality()` measures the orthogonality by checking each of the given criteria. And as shown in the output 9, we have got what we expected, as listed above, the outcome to be.

Next, we will test the orthogonality with matrices  $A$  with increasing dimensions. We will test this with the example already given in the task, that is, for  $m = n + 2$  and  $n = 1, 10, 100, 1000, 10000$ , and so on. This is also written in the code.

### Result:

I noticed that for dimension  $m = n + 2$  as  $n$  increases in each test from 1 to 10, 100, 1000, and so on, it gave the expected outcomes for all the criteria (except determinant), hence measuring the orthogonality.

But if we take  $m = n$  instead of  $m = n + 2$ , we are getting results for the determinant too. For smaller dimensions, the determinant is either 1 or -1, but as the dimensions increases, the determinant is extremely close to 1 or -1, but not exactly 1 or -1.

The main observation was that the last output it showed was for  $n = 1000$ , and then for  $n = 10000$ , the execution showed to be 'busy' and basically got stuck running the code to show any output. I needed to externally stop the program from running. This is showing us that as the dimensions get bigger, the slower and more unstable the execution becomes for the code.

Checking criteria for orthogonality:

```
import numpy as np

class Orthogonalization:
    def __init__(self, A):

        self.A = np.array(A)
```



```

        self.m, self.n = self.A.shape

    if self.m < self.n:
        raise ValueError("The number of rows m is less than the number of
                           columns n.")

    def gramschmidt(self):

        Q = np.zeros((self.m, self.n)) # Matrix of the orthonormal vectors
        R = np.zeros((self.n, self.n)) # Upper triangular matrix

        for j in range(self.n):
            v_j = self.A[:, j] # Start with the j-th column of A

            # Subtract the projection onto the previous q_i vectors
            for i in range(j):
                q_i = Q[:, i]
                R[i, j] = np.dot(q_i, v_j) # r_ij = q_i^T * a_j
                v_j = v_j - R[i, j] * q_i # v_j = v_j - r_ij * q_i

            # Compute r_jj = ||v_j||_2 (2-norm of v_j)
            R[j, j] = np.linalg.norm(v_j, 2)

            # Normalize v_j to get q_j
            Q[:, j] = v_j / R[j, j]

        return Q, R

    def checkorthogonality(Q):
        # Check if the 2-norm of each column of Q is 1
        column_norms = np.linalg.norm(Q, axis=0)
        print("2-norm of each column of Q:", column_norms)

        # Deviation of Q^T Q from the identity matrix (in 2-norm)
        I = np.eye(Q.shape[1])
        Q_T_Q = np.dot(Q.T, Q)
        deviation_norm = np.linalg.norm(Q_T_Q - I, ord=2)
        is_close_to_identity = np.allclose(Q_T_Q, I, atol=1e-8)
        print("Deviation of Q^T Q from identity (2-norm):", deviation_norm)
        print("Q^T Q is close to identity:", is_close_to_identity)

        # Eigenvalues of Q^T Q
        eigenvalues = np.linalg.eigvals(Q_T_Q)
        print("Eigenvalues of Q^T Q:", eigenvalues)

        # Determinant of Q (only if Q is square)
        if Q.shape[0] == Q.shape[1]:
            determinant = np.linalg.det(Q)
            print("Determinant of Q:", determinant)
        else:
            print("Determinant is not defined for non-square Q matrix.")

if __name__ == "__main__":
    # Create a random 3x3 matrix (m >= n)
    A = np.random.rand(3, 3)

    # Perform Gram-Schmidt orthogonalization
    orthogonalizer = Orthogonalization(A)

```

```

Q, R = orthogonalizer.gramschmidt()

# Check the orthogonality properties
checkorthogonality(Q)

```

Output:

```

2-norm of each column of Q: [1. 1. 1.]
Deviation of QT Q from identity (2-norm): 4.279244400882571e-16
QT Q is close to identity: True
Eigenvalues of QT Q: [1. 1. 1.]
Determinant of Q: 1.0

```

Code for increasing dimensions of A:

```

import numpy as np

class Orthogonalization:
    def __init__(self, A):

        self.A = np.array(A)
        self.m, self.n = self.A.shape

        if self.m < self.n:
            raise ValueError("The number of rows m is less than the number of
                               columns n.")

    def gramshmidt(self):

        Q = np.zeros((self.m, self.n)) # Matrix of the orthonormal vectors
        R = np.zeros((self.n, self.n)) # Upper triangular matrix

        for j in range(self.n):
            v_j = self.A[:, j] # Start with the j-th column of A

            # Subtract the projection onto the previous q_i vectors
            for i in range(j):
                q_i = Q[:, i]
                R[i, j] = np.dot(q_i, v_j) # r_ij = q_iT * a_j
                v_j = v_j - R[i, j] * q_i # v_j = v_j - r_ij * q_i

            # Compute r_jj = ||v_j||_2 (2-norm of v_j)
            R[j, j] = np.linalg.norm(v_j, 2)

            # Normalize v_j to get q_j
            Q[:, j] = v_j / R[j, j]

        return Q, R

    def checkorthogonality(Q):
        # Check if the 2-norm of each column of Q is 1
        column_norms = np.linalg.norm(Q, axis=0)
        print("2-norm of each column of Q:", column_norms)

        # Deviation of QT Q from the identity matrix (in 2-norm)
        I = np.eye(Q.shape[1])

```

```

Q_T_Q = np.dot(Q.T, Q)
deviation_norm = np.linalg.norm(Q_T_Q - I, ord=2)
is_close_to_identity = np.allclose(Q_T_Q, I, atol=1e-8)
print("Deviation of Q^T Q from identity (2-norm):", deviation_norm)
print("Q^T Q is close to identity:", is_close_to_identity)

# Eigenvalues of Q^T Q
eigenvalues = np.linalg.eigvals(Q_T_Q)
print("Eigenvalues of Q^T Q:", eigenvalues)

# Determinant of Q (only if Q is square)
if Q.shape[0] == Q.shape[1]:
    determinant = np.linalg.det(Q)
    print("Determinant of Q:", determinant)
else:
    print("Determinant is not defined for non-square Q matrix.")

# Testing with matrices of increasing dimensions
dimensions = [1, 10, 100, 1000, 10000]
for n in dimensions:
    m = n + 2
    print(f"\nTesting with matrix of dimensions {m} x {n}")

    # Create a random m x n matrix
    A = np.random.rand(m, n)

    # Perform Gram-Schmidt orthogonalization
    orthogonalizer = Orthogonalization(A)
    Q, R = orthogonalizer.gramschmidt()

    # Check orthogonality properties
    checkorthogonality(Q)

```

#### 10. Task:

In this Task, we will first compute the orthogonal basis of  $\text{range}(A)$  by using `scipy.linalg`'s command `qr`. Doing this means that we can remove the entire `gramschmidt()` method and instead replace it with one line of code.

Now, I have re-written the code from Task 2 but with mentioned replacement to command `qr` in the code. We observe the following by reiterating Task 1 and 2 here:

- For the orthonormal matrix, each column vector gave a 2-norm equal to 1.
- For the perfectly orthonormal matrix  $Q$ ,  $(Q^T Q - I)$  was close to zero. The 2-norm of this difference matrix gave us the measure of deviation.
- For the orthonormal matrix, all the eigenvalues of  $Q^T Q$  was equal to 1.
- For the orthonormal matrix, the determinant was very close to being either 1 or -1.
- As the dimensions increased, the execution was smoother, faster and more efficient. Unlike previously, it didn't lead to the code getting stuck.

Qualitative difference:

The QR decomposition using `scipy.linalg.qr` is more numerically stable than the manually implemented Gram-Schmidt orthogonalization, especially for matrices with larger dimensions. The command `qr` provides a built-in way to get the orthogonal basis of the  $\text{range}(A)$  more efficiently.

Gram-Schmidt is unstable:

The algorithm used in the Tasks 1 and 2 for the Gram-Schmidt orthogonalization, mathematically offers a simple route to understanding and proving various properties of QR-factorizations. But, numerically, it turns out to be less efficient and unstable because of rounding errors on a computer.

Computes orthogonal basis (Q), checks orthogonality using list of criteria from Task 2 and does so with increasing dimensions of matrix A:

```
import numpy as np
from scipy.linalg import qr

class Orthogonalization:
    def __init__(self, A):

        self.A = np.array(A)
        self.m, self.n = self.A.shape

        if self.m < self.n:
            raise ValueError("The number of rows m is less than the number of
                               columns n.")

def checkorthogonality(Q):
    # Check if the 2-norm of each column of Q is 1
    column_norms = np.linalg.norm(Q, axis=0)
    print("2-norm of each column of Q:", column_norms)

    # Deviation of Q^T Q from the identity matrix (in 2-norm)
    I = np.eye(Q.shape[1])
    Q_T_Q = np.dot(Q.T, Q)
    deviation_norm = np.linalg.norm(Q_T_Q - I, ord=2)
    is_close_to_identity = np.allclose(Q_T_Q, I, atol=1e-8)
    print("Deviation of Q^T Q from identity (2-norm):", deviation_norm)
    print("Q^T Q is close to identity:", is_close_to_identity)

    # Eigenvalues of Q^T Q
    eigenvalues = np.linalg.eigvals(Q_T_Q)
    print("Eigenvalues of Q^T Q:", eigenvalues)

    # Determinant of Q (only if Q is square)
    if Q.shape[0] == Q.shape[1]:
        determinant = np.linalg.det(Q)
        print("Determinant of Q:", determinant)
    else:
        print("Determinant is not defined for non-square Q matrix.")

# Testing with matrices of increasing dimensions
dimensions = [1, 10, 100, 1000, 10000]
for n in dimensions:
    m = n + 2
    print(f"\nTesting with matrix of dimensions {m} x {n}")

    # Create a random m x n matrix
    A = np.random.rand(m, n)

    # Perform Gram-Schmidt orthogonalization
    Q, R = qr(A, mode='full')
```

```
# Check orthogonality properties
checkorthogonality(Q)
```

#### 11. Task:

This task is based on the Householder transformations. Here, we will write a python program (as a method of previously used class `Orthogonalization`) performing this transformation. The code is given below.

In the code we can see that in the method `householder_transf()`, we first initialize  $A$ ,  $Q$  and its dimensions. We then get into a for loop. This loop goes over each column of the matrix  $A$ . The algorithm processes one column at a time to construct the upper triangular matrix  $R$  and the orthogonal matrix  $Q$ .

Then, we first extract the vector  $x$  from the  $k$ -th column of  $A$  starting from the  $k$ -th row to the end of the column. This is the portion of the column that will be transformed in the current iteration to zero out elements below the diagonal.

Next we compute the householder vector. We start by computing the 2-norm (Euclidean norm) of the vector  $x$ . This norm is used to calculate the scaling factor for the Householder vector. The 'sign' determines the sign of the first element of  $x$  to ensure numerical stability and to prevent cancellation errors during reflection.

We then create the householder vector  $v_k$  using the formula:

$$v_k = \text{sign}(x_1) * ||x||_2 * e_1 + x \quad (72)$$

`np.eye(len(x))[:, 0]` creates a standard basis vector  $e_1$  of the same length as  $x$ . `sign * norm_x * np.eye(len(x))[:, 0]` scales this basis vector, and adding it to  $x$  forms the Householder vector.

Next we normalize the vector  $v_k$  to ensure it has unit length. This is essential for forming the reflection.

We then project the submatrix of  $A$  onto the Householder vector  $v_k$ . The outer product then forms the Householder matrix used for the reflection. And then we subtract the reflected component to zero out elements in the submatrix of  $A$ . This process progressively transforms  $A$  into an upper triangular matrix  $R$ .

We finally accumulate the Householder transformations to build the orthogonal matrix  $Q$ . We do this by projecting the columns of  $Q$  onto the Householder vector  $v_k$ , forming the outer product to construct the reflection matrix and applying the reflection to the relevant columns of  $Q$ .

At the end of the loop, the matrix  $A$  has been transformed into an upper triangular matrix, which is now the matrix  $R$ .

Basically, each iteration of the loop constructs a Householder vector to zero out elements below the diagonal in the current column, transforming the matrix into an upper triangular form ( $R$ ). The product of all the Householder transformations is accumulated to form the orthogonal matrix  $Q$ .

Now let's test our code:

- Checking if  $Q$  is indeed orthogonal.  
To check if the matrix  $Q$  is orthogonal, we need to verify that the product of  $Q$  and its transpose  $Q^T$  is the identity matrix. We can do this by the second code below.
- Checking if  $A = QR$   
To check this, we only have to find the product of  $Q$  and  $R$  that we found through the code and compare with the given matrix  $A$ . This can be done using the third code under below.
- Comparing the result to Python's `scipy.linalg.qr`.  
The fourth code below is using Python's `scipy.linalg.qr`. It is observed that this is also giving the same results as the Householder transformations.

```
import numpy as np

class Orthogonalisation:
    def __init__(self, A):
        # Ensure A is an m x n numpy array
        self.A = np.array(A, dtype=float)
        self.m, self.n = self.A.shape
        if self.m < self.n:
            raise ValueError("The matrix A must have more rows than columns (m >= n)")

    def householder_transf(self):
        A = self.A.copy()
        m, n = self.m, self.n
        Q = np.eye(m)

        for k in range(n):
            # Step 1: Extract the vector x (from A[k:m, k])
            x = A[k:m, k]

            # Step 2: Compute the Householder vector v_k
            norm_x = np.linalg.norm(x)
            sign = -1 if x[0] < 0 else 1
            v_k = x + sign * norm_x * np.eye(len(x))[:, 0] # v_k = sign(x_1) * ||x||_2 * e_1 + x

            # Normalize v_k
            v_k = v_k / np.linalg.norm(v_k)

            # Step 3: Update A using the Householder transformation
            A[k:m, k:n] -= 2 * np.outer(v_k, np.dot(v_k, A[k:m, k:n]))

            # Update Q (accumulating Householder transformations)
            Q[:, k:m] -= 2 * np.outer(Q[:, k:m] @ v_k, v_k)

        # R is the modified A
        R = A
        return Q, R
```

```
# Example usage:
m=3
n=3
A = np.random.rand(m, n)
ortho = Orthogonalisation(A)
Q, R = ortho.householder_transf()
print("Q:\n", Q)
print("R:\n", R)
```

Add the following code to the above code to check if  $Q$  is orthogonal:

```
# Check orthogonality of Q
identity_check = np.allclose(np.dot(Q.T, Q), np.eye(Q.shape[0]))
print("Is Q orthogonal?", identity_check)

# Output Q^T * Q
print("Q^T * Q:")
print(np.dot(Q.T, Q))
```

Add the following code to verify that  $A = QR$ :

```
# Check if A is approximately equal to Q @ R
A_reconstructed = np.dot(Q, R)
is_equal = np.allclose(A_reconstructed, A)
print("\nIs A equal to QR?", is_equal)

# Output the original matrix A and the reconstructed QR product
print("\nOriginal matrix A:")
print(A)
print("\nReconstructed matrix QR:")
print(A_reconstructed)
```

Output:

Using `qr` command instead of the householder method:

```
import numpy as np
from scipy.linalg import qr

class Orthogonalisation:
    def __init__(self, A):
        # Ensure A is an m x n numpy array
        self.A = np.array(A, dtype=float)
        self.m, self.n = self.A.shape
        if self.m < self.n:
            raise ValueError("The matrix A must have more rows than columns (m >= n)")

# Example usage:
m=3
n=3
A = np.random.rand(m, n)
Q, R = qr(A)
print("Q:\n", Q)
print("R:\n", R)
```

```

Q:
[[-0.14009343  0.73211979  0.66661416]
 [-0.9748454  -0.21985781  0.03659211]
 [-0.17335014  0.64471944 -0.74450425]]
R:
[[-9.11457707e-01 -4.73755528e-01 -5.68797650e-01]
 [ 1.11022302e-16  9.44524239e-01  2.52738055e-01]
 [ 2.77555756e-17 -1.11022302e-16 -1.35146763e-01]]
Is Q orthogonal? True
Q^T * Q:
[[1.00000000e+00 1.73966038e-16 1.42904760e-16]
 [1.73966038e-16 1.00000000e+00 2.46477500e-16]
 [1.42904760e-16 2.46477500e-16 1.00000000e+00]]

Is A equal to QR? True

Original matrix A:
[[0.12768924 0.75787493 0.1746286 ]
 [0.88853036 0.25417736 0.49397803]
 [0.15800132 0.69107872 0.36216363]]

Reconstructed matrix QR:
[[0.12768924 0.75787493 0.1746286 ]
 [0.88853036 0.25417736 0.49397803]
 [0.15800132 0.69107872 0.36216363]]

```

```

# Check orthogonality of Q
identity_check = np.allclose(np.dot(Q.T, Q), np.eye(Q.shape[0]))
print("Is Q orthogonal?", identity_check)

# Output Q^T * Q
print("Q^T * Q:")
print(np.dot(Q.T, Q))

# Check if A is approximately equal to Q @ R
A_reconstructed = np.dot(Q, R)
is_equal = np.allclose(A_reconstructed, A)
print("\nIs A equal to QR?", is_equal)

# Output the original matrix A and the reconstructed QR product
print("\nOriginal matrix A:")
print(A)
print("\nReconstructed matrix QR:")
print(A_reconstructed)

```

## 12. Task:

In this task, a model of a signal is given by the formula:

$$y(x) = a \sin(x) + b \cos(x) + c \sin(2x) + d \cos(2x) \quad (73)$$

with unknown coefficients  $a, b, c, d$ . To determine these coefficients we are asked to take a least squares approach.

Let's assume the measured signal is stored in an array of values

$$\mathbf{y} = [y_1, y_2, \dots, y_{200}] \quad (74)$$



where each  $y_i$  is the signal at each of the 200 equidistant sampling points in  $[0, 4\pi]$ . These are given in data file `signal.dat`.

This model can be written as the system of linear equations:

$$\mathbf{y} = a\mathbf{f}_1 + b\mathbf{f}_2 + c\mathbf{f}_3 + d\mathbf{f}_4 \quad (75)$$

where,

$$\begin{aligned} \mathbf{f}_1 &= [\sin(x_1), \sin(x_2), \dots, \sin(x_{200})]^T \\ \mathbf{f}_2 &= [\cos(x_1), \cos(x_2), \dots, \cos(x_{200})]^T \\ \mathbf{f}_3 &= [\sin(2x_1), \sin(2x_2), \dots, \sin(2x_{200})]^T \\ \mathbf{f}_4 &= [\cos(2x_1), \cos(2x_2), \dots, \cos(2x_{200})]^T \end{aligned} \quad (76)$$

Thus, the signal can be expressed as a matrix form:

$$\mathbf{y} = \mathbf{A}\mathbf{F} \quad (77)$$

where  $\mathbf{y}$  is the vector of all the measured signals,  $\mathbf{F}$  is the vector of unknown coefficients  $a, b, c, d$  and  $\mathbf{A}$  is a  $200 \times 4$  matrix consisting of the RHS of equation 76.

To find the unknown coefficients  $a, b, c, d$ , we need to solve the least squares problem by minimizing the residual:

$$\arg \min_{\mathbf{F}} \|\mathbf{y} - \mathbf{A}\mathbf{F}\|_2 \quad (78)$$

The solution of this can be found through a variety of methods such as normal equations, QR-factorization, SVD (Singular Value Decomposition), etc. Here, we will solve it separately first by QR-factorization, then by SVD and compare the results along with the computational time.

The QR- factorization code and SVD code written in python is given in the code along with the output I received.

We can see that both methods gave us the same results. But comparing the computational time, we can see that the result is computed faster using the SVD method compared to QR-factorization.

Finally, I have plotted signal. There are 3 plots in one (one plotted using the measured data, one using the results from QR-factorization and one using the results from the SVD method). But, only one plot is visible as they all yield the same results. So the plots are superimposed over one another.

```
import numpy as np
import matplotlib.pyplot as plt
import time

# Load the data
data = np.loadtxt('signal.dat', delimiter=',')
x_values = data[:, 0] # First column: x values
y_values = data[:, 1] # Second column: y values (measured signal)

# Construct the design matrix A
A = np.column_stack([
```

```

    np.sin(x_values),      # First column: sin(x)
    np.cos(x_values),      # Second column: cos(x)
    np.sin(2 * x_values),  # Third column: sin(2x)
    np.cos(2 * x_values)   # Fourth column: cos(2x)
] )

### QR Factorization Approach
start_qr = time.time()

# Step 1: Compute reduced QR factorization
Q, R = np.linalg.qr(A)

# Step 2: Compute  $Q^T * b$ 
b_qr = Q.T @ y_values

# Step 3: Solve the upper triangular system  $R * x = Q^T * b$ 
x_qr = np.linalg.solve(R, b_qr)

end_qr = time.time()
time_qr = end_qr - start_qr

### SVD Approach
start_svd = time.time()

# Step 1: Compute reduced SVD,  $A = US(V^T)$ 
U, S, Vt = np.linalg.svd(A, full_matrices=False)

# Step 2: Compute  $U^T * b$ 
b_svd = U.T @ y_values

# Step 3: Solve the diagonal system  $S * w = U^T * b$ 
w_svd = b_svd / S # S is diagonal, so we can divide element-wise

# Step 4: Compute  $x = V * w$ 
x_svd = Vt.T @ w_svd

end_svd = time.time()
time_svd = end_svd - start_svd

### Comparing the results and computational time
print(f"Coefficients (QR Factorization): {x_qr}")
print(f"Coefficients (SVD): {x_svd}")
print(f"Time taken (QR Factorization): {time_qr:.6f} seconds")
print(f"Time taken (SVD): {time_svd:.6f} seconds")

# Reconstruct the signal using both methods
y_qr = A @ x_qr
y_svd = A @ x_svd

### Plotting the signal
plt.figure(figsize=(10, 6))

# Original signal (measured data)
plt.plot(x_values, y_values, label='Measured Signal', color='blue', linestyle='--')

# Reconstructed signal using QR
plt.plot(x_values, y_qr, label='Reconstructed Signal (QR)', color='green')

# Reconstructed signal using SVD

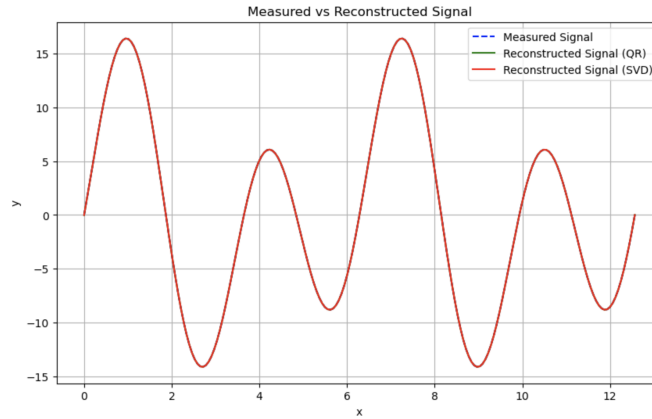
```

```
plt.plot(x_values, y_svd, label='Reconstructed Signal (SVD)', color='red')

plt.title('Measured vs Reconstructed Signal')
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.grid(True)
plt.show()
```

**Output:**

```
Coefficients (QR Factorization): [ 3.00005803  4.9998545  9.99992389 -4.99997327]
Coefficients (SVD): [ 3.00005803  4.9998545  9.99992389 -4.99997327]
Time taken (QR Factorization): 0.002957 seconds
Time taken (SVD): 0.000899 seconds
```



### 13. Task:

In this task, we will use Singular Value Decomposition (SVD) to compress an image. The image is first loaded into Python, converted into a grayscale image and then interpreted as a matrix. We then compute the SVD of this matrix and compress the image by neglecting all singular values below a chosen tolerance.

Let the grayscale image be represented by a matrix

$$A \in \mathbb{R}^{m \times n}. \quad (79)$$

Here, each entry of the matrix  $A$  represents the intensity of one pixel in the image. A darker pixel has a smaller value and a brighter pixel has a larger value.

The Singular Value Decomposition of  $A$  is given by

$$A = U \Sigma V^T, \quad (80)$$

where  $U$  and  $V$  are orthogonal matrices and  $\Sigma$  is a diagonal matrix containing the singular values

$$\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r \geq 0. \quad (81)$$

The singular values measure how much information is contained in each corresponding singular vector direction. The larger singular values contain the most important features of the image, while very small singular values usually correspond to fine details or noise.

A rank  $k$  approximation of the image is given by

$$A_k = \sum_{i=1}^k \sigma_i u_i v_i^T. \quad (82)$$

This means that instead of using all singular values, we only use the first  $k$  singular values. This gives a compressed version of the original image.

First, consider an ideal three-stripped flag such as the flag of France, Germany or Italy. If we convert the flag into grayscale, then each stripe has almost constant pixel intensity. For a vertical three-stripped flag, every row of the image matrix is the same. Therefore, all rows are linearly dependent. Similarly, for a horizontal three-stripped flag, every column is the same type of vector up to the stripe structure. Hence, for an ideal grayscale three-stripped flag, we would suggest using

$$k = 1. \quad (83)$$

This is because the exact grayscale image matrix of such an ideal flag has rank 1. In practice, if the picture has shadows, blurred edges or noise, then a slightly larger rank may be needed. But for an ideal three-stripped flag, rank 1 is enough.

Now, for the image `kvinna.jpg`, we first load the image and convert it into a grayscale matrix. We then compute the SVD from equation 80. As a first test, we reconstruct the matrix using all singular values and compute the 2-norm of the difference

$$\|A - U\Sigma V^T\|_2. \quad (84)$$

This value should be very close to zero, up to numerical round-off error.

To compress the image, we choose a tolerance `tol` and keep only the singular values satisfying

$$\sigma_i \geq \text{tol}. \quad (85)$$

If  $k$  singular values satisfy equation 85, then the compressed image is

$$A_{\text{compressed}} = \sum_{i=1}^k \sigma_i u_i v_i^T. \quad (86)$$

The quality of the compressed image depends on the tolerance. If the tolerance is small, then more singular values are kept and the image quality is better. But the compression is weaker. If the tolerance is large, then fewer singular values are kept and the image is more compressed. But the image quality becomes worse and the result may look blurry.

Thus, there is a trade-off between compression and image quality.

**Python Code:**

```

import numpy as np
import matplotlib.pyplot as plt
from matplotlib.image import imread

# Load the image

img = imread("kvinna.jpg")

# Convert the image to floating point values

img = img.astype(float)

# If the image has values between 0 and 255, scale them to [0, 1]

if img.max() > 1:
    img = img / 255.0

# Convert RGB image to grayscale

# If the image has 3 colour channels, we use the standard grayscale formula

if img.ndim == 3:
    # If there is an alpha channel, ignore it
    img = img[:, :, :3]

    '''
    A = (
        0.299 * img[:, :, 0]
        \+ 0.587 * img[:, :, 1]
        \+ 0.114 * img[:, :, 2]
    )
    '''

else:
    A = img

print("Shape of grayscale image matrix A:", A.shape)

# Perform SVD of the image matrix

U, S, Vt = np.linalg.svd(A, full_matrices=False)

# Reconstruct the image using all singular values

A_reconstructed = (U * S) @ Vt

# Compute the 2-norm of the reconstruction error

error_norm = np.linalg.norm(A - A_reconstructed, 2)

print("2-norm of A - U Sigma V^T:", error_norm)

# Choose different tolerances for compression

tolerances = [
    0.20 * S[0],
    0.10 * S[0],
    0.05 * S[0],

```

```

0.01 * S[0]
]

# Plot the original image first

plt.figure(figsize=(6, 6))
plt.imshow(A, cmap="gray")
plt.title("Original grayscale image")
plt.axis("off")
plt.show()

# Compress and plot the image for each tolerance

for tol in tolerances:
    # Keep singular values greater than or equal to tolerance
    k = np.sum(S >= tol)

    '''
    # Construct the compressed image
    A_compressed = (U[:, :k] * S[:, :k]) @ Vt[:, :k, :]

    # Clip values to stay in the interval [0, 1]
    A_compressed = np.clip(A_compressed, 0, 1)

    # Compute the compression error
    compression_error = np.linalg.norm(A - A_compressed, 2)

    print("\nTolerance:", tol)
    print("Number of singular values kept:", k)
    print("2-norm compression error:", compression_error)

    # Visualize the compressed image
    plt.figure(figsize=(6, 6))
    plt.imshow(A_compressed, cmap="gray")
    plt.title(f"Compressed image, rank = {k}")
    plt.axis("off")
    plt.show()
    '''

```

When the code is run, it prints the shape of the grayscale matrix, the 2-norm reconstruction error using the full SVD, and the number of singular values kept for each tolerance.

From the full SVD reconstruction, the value of  $\|A - U\Sigma V^T\|_2$  is expected to be very close to zero. This confirms that the SVD decomposition has reconstructed the original image correctly, up to numerical round-off error.

When we increase the tolerance, fewer singular values are kept. Therefore, the rank of the compressed approximation decreases. This gives stronger compression, but the quality of the image becomes worse. The image starts losing finer details and becomes more blurred.

When we decrease the tolerance, more singular values are kept. Therefore, the compressed image is closer to the original image. The quality improves, but less compression is achieved.

Thus, the tolerance controls the balance between compression and image quality. A small tolerance gives better quality, while a large tolerance gives stronger compression.

#### 14. Task:

The condition number of a matrix can give a sharp estimate of the sensitivity of  $x$  with respect to

perturbations of  $b$  when solving  $Ax = b$ , this means there exists a right hand side  $b$  and a perturbation  $\delta b$  such that

$$\frac{\|\delta x\|_2}{\|x\|_2} = \kappa_2(A) \frac{\|\delta b\|_2}{\|b\|_2} \quad (87)$$

In this task, we need to find a vector pair  $(b, \delta b)$  for which this equality holds.

To find this, we need to consider the SVD (Singular Value Decomposition) of the matrix  $A$ .

We know the matrix  $A$  can be decomposed as  $A = U\Sigma V^T$  where,  $U$  is an orthogonal matrix with left singular vectors  $u_1, u_2, \dots, u_n$ ,  $V$  is an orthogonal matrix with right singular vectors  $v_1, v_2, \dots, v_n$ , and  $\Sigma$  is a diagonal matrix of the singular values  $\sigma_1, \sigma_2, \dots, \sigma_n$  such that,  $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_n \geq 0$ .

The condition number of a matrix  $A$  with respect to 2-norm is given by:

$$\kappa(A) = \frac{\sigma_1}{\sigma_n} \quad (88)$$

where  $\sigma_1$  is the largest singular value and  $\sigma_n$  is the smallest singular value.

Next, to find vectors  $b$  and  $\delta b$ , we can express them in terms of the left singular vectors of  $A$ .

Let  $b = u_1$ , where  $u_1$  is the left singular vector corresponding to the largest singular value  $\sigma_1$  and let  $\delta b = u_n$ , where  $u_n$  is the left singular vector corresponding to the smallest singular value  $\sigma_n$ .

This is because the perturbation  $\delta b$  is chosen such that it would be in the direction in which  $A$  is least sensitive while  $b$  is chosen such that it would be in the direction in which  $A$  is most sensitive.

We can verify the choice:

For  $b$ ,  $Ax = b = u_1$ .

Since  $Av_1 = \sigma_1 v_1$ , then  $x = \frac{1}{\sigma_1} v_1$ .

So,  $\|x\|_2 = \frac{1}{\sigma_1}$

For  $\delta b$ ,  $A\delta x = \delta b = u_n$ .

Since  $Av_n = \sigma_n v_n$ , then  $\delta x = \frac{1}{\sigma_n} v_n$ .

So,  $\|\delta x\|_2 = \frac{1}{\sigma_n}$

Thus,

In the equation  $\frac{\|\delta x\|_2}{\|x\|_2} = \kappa_2(A) \frac{\|\delta b\|_2}{\|b\|_2}$ ,

LHS:

$$\frac{\|\delta x\|_2}{\|x\|_2} = \frac{\frac{1}{\sigma_n}}{\frac{1}{\sigma_1}} = \frac{\sigma_1}{\sigma_n} = \kappa(A) \quad (89)$$

RHS:

$$\kappa_2(A) \frac{\|\delta b\|_2}{\|b\|_2} = \kappa_2(A) \frac{\|u_n\|_2}{\|u_1\|_2} = \kappa_2(A)(1) = \kappa_2(A) \quad (90)$$

Thus, LHS = RHS.

Hence, we have verified that the choice  $b = u_1$  and  $\delta b = u_n$  where  $u_1$  and  $u_n$  are the left singular vectors corresponding to the largest and smallest singular values respectively, satisfies the equality.

#### 15. **Task:**

This task is regarding Hilbert matrices.

In the code below, we define a  $50 \times 50$  Hilbert matrix  $H$  along with the condition number of the matrix  $H$ .

We know that  $\kappa(H) = \frac{\sigma_1}{\sigma_n}$  where  $\sigma_1$  and  $\sigma_n$  are the largest and smallest singular values of  $H$ .

The condition number ( $\kappa_2(H)$ ) tells us how sensitive the solution  $x$  is to changes in  $b$  when solving  $Ax = b$ . For ill-conditioned matrices like the Hilbert matrix, this number is very large. Here, we find the 2-norm condition number of the Hilbert matrix.

Worst-case  $b$  and  $\delta b$ :

- $b = u_1$  (first left singular vector): This corresponds to  $Hv_1 = \sigma_1 u_1$
- $\delta b = u_n$  (last left singular vector): This corresponds to  $Hv_n = \sigma_n u_n$

Using the fact that  $H$  is symmetric and positive definite, the right singular vectors ( $v$ ) are the same as the left singular vectors ( $u$ ).  $v_1$  and  $v_n$  are approximately aligned with the first and last column of the inverse of  $H$ .

We set  $b$  to the first column of the inverse Hilbert matrix, which approximates the vector corresponding to the largest singular value (associated with  $u_1$ , the left singular vector corresponding to  $\sigma_1$ ). And, we set  $\delta b$  to the last column of the inverse Hilbert matrix, which approximates the vector corresponding to the smallest singular value (associated with  $u_n$ , the left singular vector corresponding to  $\sigma_n$ ).

We then solve two linear systems:

$$\begin{aligned} Hx &= b \\ H\delta x &= \delta b \end{aligned} \quad (91)$$

We finally find  $\frac{\|\delta x\|_2}{\|x\|_2}$  and  $\kappa_2(H) \frac{\|\delta b\|_2}{\|b\|_2}$ . The result can be viewed below.



From the previous task, we know that there exists vectors  $b$  and  $\delta b$  such that:

$$\frac{\|\delta x\|_2}{\|x\|_2} = \kappa_2(H) \frac{\|\delta b\|_2}{\|b\|_2} \quad (92)$$

This means that in the worst-case scenario, the relative change in  $x$  due to a perturbation in  $b$  is amplified by the condition number  $\kappa_2(H)$ . The goal is to check whether this relationship holds using the Hilbert matrix  $H$ .

We have got:

$$\begin{aligned} \frac{\|\delta x\|_2}{\|x\|_2} &= 2.86 \times 10^{28} \\ \kappa_2(H) &= 2.31 \times 10^{19} \quad \text{and hence,} \\ \kappa_2(H) \times \frac{\|\delta b\|_2}{\|b\|_2} &= 4.90 \times 10^{47} \end{aligned} \quad (93)$$

Comparing the values of  $\frac{\|\delta x\|_2}{\|x\|_2}$  and  $\kappa_2(H) \frac{\|\delta b\|_2}{\|b\|_2}$ , we can observe that these two quantities are vastly different, with the right-hand side of 92 being much larger than the left-hand side.

Thus, the result from the code does not verify the strict equality from the previous task.

```
import numpy as np
from scipy.linalg import hilbert, invhilbert

# Define the size of the Hilbert matrix
n = 50

# Construct the Hilbert matrix and its inverse
H = hilbert(n)
H_inv = invhilbert(n)

# Compute the condition number of the Hilbert matrix in the 2-norm
cond_H = np.linalg.cond(H, 2)

# Define worst-case b and delta_b
b = H_inv[:, 0]
delta_b = H_inv[:, -1]

# Solve the systems Hx = b and H(delta_x) = delta_b
x = np.linalg.solve(H, b)
delta_x = np.linalg.solve(H, delta_b)

# Compute the relative perturbations in x and b
relative_x_perturbation = np.linalg.norm(delta_x) / np.linalg.norm(x)
relative_b_perturbation = np.linalg.norm(delta_b) / np.linalg.norm(b)

# Expected worst-case amplification
worst_case_amplification = cond_H * relative_b_perturbation

relative_x_perturbation, worst_case_amplification, cond_H
```

**Output:****16. Task:**

In this task we need to show that a strictly diagonally dominant matrix  $A$  is invertible. The idea is to show that there is no  $u \neq 0$  which satisfies  $Au = 0$ .

Let's first assume that  $Au = 0$  for some  $u = [u_1, u_2, \dots, u_n]^T \neq 0$  where  $A \in \mathbb{R}^{n \times n}$  is a strictly diagonally dominant matrix.

This implies that,

$$\sum_{j=1}^n a_{ij}u_j = 0 \quad (94)$$

or,

$$a_{ii}u_i + \sum_{\substack{j=1 \\ j \neq i}}^n a_{ij}u_j = 0 \quad (95)$$

Since we assumed that  $u \neq 0$ , then at least one element of  $u$  is non-zero.

If all elements  $u_i$  of  $u$  had the same absolute value,  $|u_1| = |u_2| = \dots = |u_n|$ , then for each row  $i$ ,

$$\begin{aligned} a_{ii}u_i + \sum_{\substack{j=1 \\ j \neq i}}^n a_{ij}u_j &= 0 \\ |a_{ii}||u_i| &= \left| \sum_{\substack{j=1 \\ j \neq i}}^n a_{ij}u_j \right| && \text{Taking absolute value} \\ |a_{ii}||u_i| &\leq \sum_{\substack{j=1 \\ j \neq i}}^n |a_{ij}||u_j| && \text{Triangle inequality} \\ |a_{ii}||u_i| &\leq |u_j| \sum_{\substack{j=1 \\ j \neq i}}^n |a_{ij}| && |u_i| = |u_j| \text{ for all } j \\ |a_{ii}| &\leq \sum_{\substack{j=1 \\ j \neq i}}^n |a_{ij}| \end{aligned} \quad (96)$$

But, this contradicts the strict diagonal dominance condition where  $|a_{ii}| > \sum_{\substack{j=1 \\ j \neq i}}^n |a_{ij}|$ .

Thus, it is not possible for all elements of  $u$  to have the same absolute value.

Next, if there is an index  $k$  such that  $|u_k| \geq |u_j|$  for all  $j$ . WLOG we'll assume  $|u_k| > 0$  as  $u \neq 0$ . Now similarly for each row  $k$ ,

$$\begin{aligned}
a_{kk}u_k + \sum_{\substack{j=1 \\ j \neq k}}^n a_{kj}u_j &= 0 \\
|a_{kk}||u_k| &= \left| \sum_{\substack{j=1 \\ j \neq k}}^n a_{kj}u_j \right| && \text{Taking absolute value} \\
|a_{kk}||u_k| &\leq \sum_{\substack{j=1 \\ j \neq k}}^n |a_{kj}||u_j| && \text{Triangle inequality} \\
|a_{kk}||u_k| &\leq |u_k| \sum_{\substack{j=1 \\ j \neq k}}^n |a_{kj}| && |u_k| \leq |u_j| \text{ for all } j \neq k \\
|a_{kk}| &\leq \sum_{\substack{j=1 \\ j \neq k}}^n |a_{kj}|
\end{aligned} \tag{97}$$

But, this also contradicts the strict diagonal dominance condition where  $|a_{kk}| > \sum_{\substack{j=1 \\ j \neq k}}^n |a_{kj}|$ .

Thus, it is not possible for any element  $u_i$  of  $u$  to satisfy  $|u_k| \geq |u_j|$  for all  $j$ .

Hence, we have shown that there can't be any  $u_i$  in  $u$  such that  $|u_k| \geq |u_j|$  for all  $j$  and neither can all  $u_i$  be the same absolute value.

This implies that there is no such  $u \neq 0$  which satisfies  $Au = 0$ . Or, the only solution this has is  $u = 0$ . This is a contradiction.

This would mean that  $A$  has no non-trivial null space and thus,  $A$  is invertible.

#### 17. Task:

This task is about discretizing the given differential equation  $-u''(x) = f(x)$  over  $[0, 1]$  with boundary conditions  $u(0) = u(1) = 0$ .

To discretize the equation, we approximate the second derivative using finite differences. This is done using the central difference formula. For  $n + 1$  equidistant points  $x_i = \frac{i}{n}$ , the second derivative at  $x_i$  can be approximated as:

$$-u''(x_i) \approx \frac{u_{i+1} - 2u_i + u_{i-1}}{\Delta x^2} \tag{98}$$

where  $\Delta x = \frac{1}{n}$ . Hence,

$$-\frac{u_{i+1} - 2u_i + u_{i-1}}{\Delta x^2} = f(x_i) \tag{99}$$

Or,

$$u_{i-1} - 2u_i + u_{i+1} = -\Delta x^2 f(x_i) \quad (100)$$

Next, we see that the equation 100 forms a linear system for  $i = 1, 2, \dots, n-1$ . The boundary conditions  $u(0) = u(1) = 0$  imply that  $u_0 = u_n = 0$ . Hence, the linear system can be written as:

$$\begin{aligned} -2u_1 + u_2 &= -\Delta x^2 f(x_1) \quad i = 1 \\ u_{i-1} - 2u_i + u_{i+1} &= -\Delta x^2 f(x_i) \quad i = 2, \dots, n-2 \\ u_{n-2} - 2u_{n-1} &= -\Delta x^2 f(x_{n-1}) \end{aligned} \quad (101)$$

This system can be re-written as:

$$\begin{bmatrix} -2 & 1 & 0 & \cdots & 0 \\ 1 & -2 & 1 & \cdots & 0 \\ 0 & 1 & -2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \cdots & \vdots \\ 0 & 0 & \cdots & 1 & -2 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ \vdots \\ u_{n-1} \end{bmatrix} = \begin{bmatrix} -\Delta x^2 f(x_1) \\ -\Delta x^2 f(x_2) \\ -\Delta x^2 f(x_3) \\ \vdots \\ -\Delta x^2 f(x_{n-1}) \end{bmatrix} \quad (102)$$

Hence the matrix representing this system is:

$$\begin{bmatrix} -2 & 1 & 0 & \cdots & 0 \\ 1 & -2 & 1 & \cdots & 0 \\ 0 & 1 & -2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \ddots & \vdots \\ 0 & 0 & \cdots & 1 & -2 \end{bmatrix}.$$

This is a tri-diagonal matrix. Not only is it tri-diagonal, but it is diagonally dominant. The determinant of a diagonally dominant tri-diagonal matrix with non-zero diagonal elements is non-zero. Hence, this matrix is **non-singular**.

Finally the matrix that is given to us in the task,

$$\begin{bmatrix} 1 & -1 & 0 & & \\ -1 & 2 & -1 & \ddots & 0 \\ 0 & \ddots & \ddots & \ddots & 0 \\ & \ddots & -1 & 2 & 1 \\ & & 0 & -1 & 1 \end{bmatrix} \quad (103)$$

This is also a diagonally dominant tri-diagonal matrix with non-zero diagonal elements. Hence, its determinant is also non-zero.

Thus, this matrix is also **non-singular**.

#### 18. Task:

In this task, we will first find the rank of a matrix, say,  $M = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$ , by implementing the LU-decomposition, QR-decomposition and Singular Value Decomposition(SVD).

Determining rank of a matrix using LU-Decomposition:

In the Python code given, we have used the built-in method `lu(A)` from `scipy.linalg`. We then tested this method on the matrix  $M$ . We have found the rank to be **2**.

Determining rank of a matrix using QR-Decomposition:

In the Python code given, we have used the built-in method `np.linalg.qr(A)`. We then tested this method on the matrix  $M$ . We have found the rank to be **2**.

Determining rank of a matrix using Singular Value Decomposition(SVD):

In the Python code given, we have used the built-in method `np.linalg.svd(A)`. We then tested this method on the matrix  $M$ . We have found the rank to be **2**.

We can observe that all three methods is giving the same rank for the matrix  $M$ . Now, we will determine the rank of the Kahan Matrix using these three methods.

We will first determine the rank using these methods of the Kahan Matrix when it's dimension,  $n = 5$  and  $\theta = 0.5$ . As see in the output, we can see that the rank is **5** using all the three methods.

Now, setting  $\theta = 1.2$ , we'll slowly increase  $n$ . The following are my observations:

| n        | Rank using LU | Rank using QR | Rank using SVD |
|----------|---------------|---------------|----------------|
| 5        | 5             | 5             | 5              |
| 10       | 10            | 10            | 10             |
| 15       | 15            | 15            | 15             |
| 20       | 20            | 20            | 20             |
| 25       | 25            | 25            | 25             |
| 30       | 30            | 30            | 30             |
| 35       | 35            | 35            | 35             |
| 36       | 36            | 36            | 36             |
| 37       | 37            | 37            | 37             |
| 38       | 38            | 38            | 38             |
| 39       | 39            | 39            | 39             |
| 40       | 40            | 40            | 39             |
| 45       | 45            | 45            | 44             |
| 50       | 50            | 50            | 49             |
| $\vdots$ | $\vdots$      | $\vdots$      | $\vdots$       |
| 90       | 90            | 90            | 89             |

It can be seen that around  $n = 40$  onwards, the rank of the Kahan Matrix found using SVD is not equal to the ranks of the matrix with the same dimension found by the LU and QR decomposition methods.

Possible reasons:

- (a) The SVD decomposition computes singular values. For large matrices, some singular values can be extremely small. Small singular values may be treated as zero due to the machine precision or because they fall below a chosen threshold. This is more common when the matrix has nearly linearly dependent columns, which is possible with larger  $n$  values.
- (b) With  $n = 40$  and  $\theta = 1.2$ , the matrix is likely close to rank-deficiency, causing the smallest singular value to fall below the threshold used in the rank calculation. LU and QR decomposition methods, do not explicitly compute singular values and thus may not exhibit the same sensitivity to small numerical errors like SVD.
- (c) The Kahan matrix could be ill-conditioned for certain parameters. SVD is more likely to "detect" this ill-conditioning through small singular values, leading to a reduction in the computed rank.
- (d) As  $n$  increases, the matrix size grows, and numerical round-off errors accumulate, particularly for methods that rely on floating-point precision. The LU and QR methods typically use pivoting to remove these errors, but SVD reveals numerical deficiencies through the singular values.

### Python Code:

```
import numpy as np
from scipy.linalg import lu

# Define the matrix A
A = np.array([[1, 2, 3],
              [4, 5, 6],
              [7, 8, 9]])

# Perform LU decomposition
P, L, U = lu(A)

# Count the number of non-zero rows in U
rank = np.sum(np.abs(U.diagonal()) > 1e-12)

print(f"Rank of the matrix: {rank}")
```

### Output:

```
Rank of the matrix: 2
```

### Python Code:

```
import numpy as np

# Define the matrix A
A = np.array([[1, 2, 3],
              [4, 5, 6],
              [7, 8, 9]])

# Perform QR decomposition
```

```

Q, R = np.linalg.qr(A)

# Count the number of non-zero diagonal elements in R
rank = np.sum(np.abs(np.diag(R)) > 1e-12)

print("Matrix A:")
print(A)
print("\nMatrix Q:")
print(Q)
print("\nMatrix R:")
print(R)
print(f"\nRank of the matrix: {rank}")

```

**Output:**

```

Matrix A:
[[1 2 3]
 [4 5 6]
 [7 8 9]]

Matrix Q:
[[-0.12309149  0.90453403  0.40824829]
 [-0.49236596  0.30151134 -0.81649658]
 [-0.86164044 -0.30151134  0.40824829]]

Matrix R:
[[-8.12403840e+00 -9.60113630e+00 -1.10782342e+01]
 [ 0.00000000e+00  9.04534034e-01  1.80906807e+00]
 [ 0.00000000e+00  0.00000000e+00 -8.88178420e-16]]

Rank of the matrix: 2

```

**Python Code:**

```

import numpy as np

# Define the matrix A
A = np.array([[1, 2, 3],
              [4, 5, 6],
              [7, 8, 9]])

# Perform SVD decomposition
U, S, Vt = np.linalg.svd(A)

# Count the number of non-zero singular values in S
rank = np.sum(S > 1e-12)

print("Matrix A:")
print(A)
print("\nSingular values (S):")
print(S)
print(f"\nRank of the matrix: {rank}")

```

**Output:**

Matrix A:

```
[[1 2 3]
 [4 5 6]
 [7 8 9]]
```

Singular values (S):

```
[1.68481034e+01 1.06836951e+00 4.41842475e-16]
```

Rank of the matrix: 2

## Python Code:

```
import numpy as np

# Function to construct the Kahan matrix A
def kahan_matrix(n, theta):
    s = np.sin(theta)
    c = np.cos(theta)

    # Step 1: Construct diagonal matrix S
    powers_of_s = np.array([s**i for i in range(n)])
    S = np.diag(powers_of_s)

    # Step 2: Construct upper triangular matrix R
    R = np.triu(-c * np.ones((n, n)), k=1) + np.eye(n)

    # Step 3: Compute (SR)
    SR = np.dot(S, R)

    # Step 4: Compute A = (SR)^T * (SR)
    A = np.dot(SR.T, SR)

    return A

n = 5 # Matrix size
theta = 0.5 # Parameter theta

# Generate the Kahan matrix
A = kahan_matrix(n, theta)

# Print the resulting matrix A
print("Kahan Matrix A:")
print(A)

from scipy.linalg import lu, qr, svd

# --- LU Decomposition ---
P, L, U = lu(A)
rank_lu = np.sum(np.abs(U.diagonal()) > 1e-12)

# --- QR Decomposition ---
Q, R = qr(A)
rank_qr = np.sum(np.abs(np.diag(R)) > 1e-12)

# --- SVD Decomposition ---
```



```

U_svd, S_svd, Vt = svd(A)
rank_svd = np.sum(S_svd > 1e-12)

# Print the ranks
print(f"\nRank (LU method): {rank_lu}")
print(f"Rank (QR method): {rank_qr}")
print(f"Rank (SVD method): {rank_svd}")

```

## Output:

```

Kahan Matrix A:
[[ 1.          -0.87758256 -0.87758256 -0.87758256 -0.87758256]
 [-0.87758256  1.          0.56843981  0.56843981  0.56843981]
 [-0.87758256  0.56843981  1.          0.90080639  0.90080639]
 [-0.87758256  0.56843981  0.90080639  1.          0.97720046]
 [-0.87758256  0.56843981  0.90080639  0.97720046  1.          ]]

Rank (LU method): 5
Rank (QR method): 5
Rank (SVD method): 5

```

## Python Code:

```

import numpy as np

# Function to construct the Kahan matrix A
def kahan_matrix(n, theta):
    s = np.sin(theta)
    c = np.cos(theta)

    # Step 1: Construct diagonal matrix S
    powers_of_s = np.array([s**i for i in range(n)])
    S = np.diag(powers_of_s)

    # Step 2: Construct upper triangular matrix R
    R = np.triu(-c * np.ones((n, n)), k=1) + np.eye(n)

    # Step 3: Compute (SR)
    SR = np.dot(S, R)

    # Step 4: Compute A = (SR)^T * (SR)
    A = np.dot(SR.T, SR)

    return A

n = 90 # Matrix size
theta = 1.2 # Parameter theta

# Generate the Kahan matrix
A = kahan_matrix(n, theta)

# Print the resulting matrix A
print("Kahan Matrix A:")
print(A)

from scipy.linalg import lu, qr, svd

```

```

# --- LU Decomposition ---
P, L, U = lu(A)
rank_lu = np.sum(np.abs(U.diagonal()) > 1e-12)

# --- QR Decomposition ---
Q, R = qr(A)
rank_qr = np.sum(np.abs(np.diag(R)) > 1e-12)

# --- SVD Decomposition ---
U_svd, S_svd, Vt = svd(A)
rank_svd = np.sum(S_svd > 1e-12)

# Print the ranks
print(f"\nRank (LU method): {rank_lu}")
print(f"Rank (QR method): {rank_qr}")
print(f"Rank (SVD method): {rank_svd}")

```

**Output:**

```

Kahan Matrix A:
[[ 1.          -0.36235775 -0.36235775 ... -0.36235775 -0.36235775
  -0.36235775]
 [-0.36235775  1.          -0.1834759 ... -0.1834759 -0.1834759
  -0.1834759 ]
 [-0.36235775 -0.1834759  1.          ... -0.0280818 -0.0280818
  -0.0280818 ]
 ...
 [-0.36235775 -0.1834759 -0.0280818 ... 1.          0.99999346
  0.99999346]
 [-0.36235775 -0.1834759 -0.0280818 ... 0.99999346 1.
  0.99999432]
 [-0.36235775 -0.1834759 -0.0280818 ... 0.99999346 0.99999432
  1.          ]]

Rank (LU method): 90
Rank (QR method): 90
Rank (SVD method): 89

```

## 19. Task:

There are lots of "islands" in the world-wide-web, meaning clusters of websites that are not connected to other parts of the world-wide-web via hyperlinks. Assume that there are  $r$  different clusters. We want to prove that the dimension of the eigenspace corresponding to the eigenvalue 1 of the hyperlink matrix  $H$  is at least  $r$ .

Suppose that the web graph consists of  $r$  disconnected clusters. Since there are no hyperlinks between the different clusters, we can reorder the webpages so that the hyperlink matrix  $H$  becomes a block diagonal matrix of the form

$$H = \begin{bmatrix} H_1 & 0 & \cdots & 0 & 0 & H_2 & \cdots & 0 & \vdots & \vdots & \ddots & \vdots & 0 \\ 0 & \cdots & H_r & & & & & & & & & & \end{bmatrix}. \quad (104)$$

Here, each matrix  $H_i$  is the hyperlink matrix corresponding to the  $i$ -th disconnected cluster. Since each cluster is disconnected from the others, a page in one cluster only links to pages inside the same cluster. Therefore, each block  $H_i$  is itself a hyperlink matrix. In particular, each column of  $H_i$  sums to 1. Hence, each  $H_i$  is a column stochastic matrix. This means that

$$\mathbf{1}^T H_i = \mathbf{1}^T, \quad (105)$$

where  $\mathbf{1}$  is the vector whose entries are all equal to 1. Equation 117 shows that 1 is an eigenvalue of  $H_i^T$ . Since a matrix and its transpose have the same eigenvalues, it follows that 1 is also an eigenvalue of  $H_i$ . Therefore, for each block  $H_i$ , there exists a non-zero vector  $v_i$  such that

$$H_i v_i = v_i. \quad (106)$$

Now we construct eigenvectors of the full matrix  $H$ . Define the vector  $w_1$  by putting  $v_1$  in the first block and zeros everywhere else:

$$w_1 = \begin{bmatrix} v_1 & 0 & \vdots & 0 \end{bmatrix}. \quad (107)$$

$$w_1. \quad (108)$$

Hence,  $w_1$  is an eigenvector of  $H$  corresponding to the eigenvalue 1. Similarly, for each cluster  $i = 1, 2, \dots, r$ , we can define

$$w_i = \begin{bmatrix} 0 & \vdots & v_i & \vdots & 0 \end{bmatrix}, \quad (109)$$

where  $v_i$  is placed in the  $i$ -th block. Using the same argument as in equation 117, we obtain

$$H w_i = w_i, \quad i = 1, 2, \dots, r. \quad (110)$$

Thus,  $w_1, w_2, \dots, w_r$  are all eigenvectors of  $H$  corresponding to the eigenvalue 1. Also, these vectors are linearly independent because they have non-zero entries in different blocks. For example,  $w_1$  is non-zero only in the first cluster,  $w_2$  is non-zero only in the second cluster, and so on. Therefore, the eigenspace of  $H$  corresponding to the eigenvalue 1 contains at least  $r$  linearly independent eigenvectors. Hence,

$$\dim E_1(H) \geq r, \quad (111)$$

where  $E_1(H)$  denotes the eigenspace of  $H$  corresponding to the eigenvalue 1. Therefore, we have proved that if the web graph has  $r$  disconnected clusters, then the dimension of the eigenspace corresponding to the eigenvalue 1 of the hyperlink matrix  $H$  is at least  $r$ .

For the disconnected hyperlink graph in the task, the hyperlink matrix is

$$H = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & \frac{1}{2} & \frac{1}{2} & 0 & 0 & \frac{1}{2} & 0 & \frac{1}{2} & 0 & 0 & \frac{1}{2} & \frac{1}{2} \\ 0 & & & & & & & & & & & \end{bmatrix}. \quad (112)$$

This matrix has two disconnected blocks:

$$H_1 = \begin{bmatrix} 0 & 1 & 1 & 0 \end{bmatrix} \quad (113)$$

and

$$H_2 = \begin{bmatrix} 0 & \frac{1}{2} & \frac{1}{2} & \frac{1}{2} & 0 & \frac{1}{2} & \frac{1}{2} & \frac{1}{2} & 0 \end{bmatrix}. \quad (114)$$

$$\begin{bmatrix} 1 & 1 \end{bmatrix}. \quad (115)$$

$$\begin{bmatrix} 1 & 1 & 1 \end{bmatrix}. \quad (116)$$

Therefore, for the full matrix  $H$ , two eigenvectors corresponding to the eigenvalue 1 are

$$w_1 = \begin{bmatrix} 1 & 1 & 0 & 0 & 0 \end{bmatrix}, \quad w_2 = \begin{bmatrix} 0 & 0 & 1 & 1 & 1 \end{bmatrix}. \quad (117)$$

These two vectors are clearly linearly independent. Hence, in this example, the eigenspace corresponding to the eigenvalue 1 has dimension at least 2. This agrees with the fact that there are  $r = 2$  disconnected clusters. **Python Code:**

```

import numpy as np
Define the hyperlink matrix H
H = np.array([
[0, 1, 0, 0, 0],
[1, 0, 0, 0, 0],
[0, 0, 0, 1/2, 1/2],
[0, 0, 1/2, 0, 1/2],
[0, 0, 1/2, 1/2, 0]
], dtype=float)
Compute eigenvalues and eigenvectors
eigenvalues, eigenvectors = np.linalg.eig(H)
print("Eigenvalues of H:")
print(eigenvalues)
Check the two eigenvectors corresponding to eigenvalue 1
w1 = np.array([1, 1, 0, 0, 0], dtype=float)
w2 = np.array([0, 0, 1, 1, 1], dtype=float)
print("\nH w1:")
print(H @ w1)
print("\nw1:")
print(w1)
print("\nH w2:")
print(H @ w2)
print("\nw2:")
print(w2)
Check linear independence
W = np.column_stack((w1, w2))
rank_W = np.linalg.matrix_rank(W)
print("\nRank of the matrix with columns w1 and w2:")
print(rank_W)

```

```

Eigenvalues of H:
[ 1. -1. -0.5 1. -0.5]
H w1:
[1. 1. 0. 0. 0.]
w1:
[1. 1. 0. 0. 0.]
H w2:
[0. 0. 1. 1. 1.]
w2:
[0. 0. 1. 1. 1.]
Rank of the matrix with columns w1 and w2:
2

```

From the Python output, we can see that the eigenvalue 1 appears twice for the given matrix  $H$ . We also checked directly that

$$Hw_1 = w_1$$

and

$$Hw_2 = w_2.$$

Since the rank of the matrix with columns  $w_1$  and  $w_2$  is 2, the two eigenvectors are linearly independent. Thus, for this example, the eigenspace corresponding to the eigenvalue 1 has dimension at least 2, which matches the number of disconnected clusters in the graph.

## 20. Task:

In this task, if  $A$  is an  $m \times n$  matrix, we need to show that,

$$N(A^T)^\perp = \text{Range}(A) \quad (118)$$

This means we have to just show that  $N(A^T) \perp \text{Range}(A)$ . Or, that the null space of the transpose of the matrix  $A$ , is orthogonal to the range, or the column space of  $A$ .

Let's consider a  $\mathbf{u} \in \text{Range}(A)$  and a  $\mathbf{v} \in N(A^T)$ .

By the definition of  $\text{Range}(A)$ , there exists a  $x \in \mathbb{R}^n$  such that  $\mathbf{u} = Ax$ .  
By the definition of  $N(A^T)$ , we know that  $A^T \mathbf{v} = 0$ .

Let us compute  $\mathbf{u}^T \mathbf{v}$ .

$$\begin{aligned} \mathbf{u}^T \mathbf{v} &= (Ax)^T \mathbf{v} \\ &= x^T A^T \mathbf{v} \\ &= x^T (A^T \mathbf{v}) \\ &= x^T (0) \\ &= 0 \end{aligned} \quad (119)$$

So,  $\mathbf{u}^T \mathbf{v} = 0$  for arbitrary  $\mathbf{u}$  and  $\mathbf{v}$ .

Hence,  $\mathbf{u} \perp \mathbf{v}$ .

$\implies N(A^T) \perp \text{Range}(A)$  or,  $N(A^T)^\perp = \text{Range}(A)$ .

## 21. Task:

In this task we need to prove that if  $A \in \mathbb{R}^{m \times m}$  is a nonsingular normal matrix and  $\mathbf{b}$  is

$$\mathbf{b} = \sum_{l=1}^k \gamma_l \mathbf{v}_l \quad (120)$$

Then, the GMRES (Generalized Minimal Residual) algorithm will terminate in at most  $k$  iterations when solving  $Ax = \mathbf{b}$  with the initial guess  $x_0 = 0$ .

First, we know that  $A$  is normal. That is,  $A^T A = A A^T$ .

This implies that  $A$  has an orthogonal set of eigenvectors  $\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_m$  and hence can be diagonalized as  $A = V \Lambda V^T$ , where  $V$  is the matrix whose columns are the eigenvectors of  $A$ ,  $\Lambda$  is the diagonal matrix of the corresponding eigenvalues.

Next, We are given that  $\mathbf{b} = \sum_{l=1}^k \gamma_l \mathbf{v}_l$ . Here  $\mathbf{v}_l$  are the eigenvectors corresponding to eigenvalues  $\lambda_l$  of  $A$  and  $\gamma_l$  are it's scalar coefficients.

The GMRES is an iterative method for solving linear systems. It minimizes the residual  $r_i = \mathbf{b} - Ax_i$  over a Krylov subspace at each iteration. That is, at an iteration  $i$ ,  $x_i$  lies in the Krylov subspace

$\mathcal{K}_i(A, r_0) = \text{span}\{r_0, Ar_0, A^2r_0, \dots, A^{i-1}r_0\}$ . Since  $x_0 = 0$ ,  $r_0 = \mathbf{b}$ .

In GMRES, the algorithm seeks an approximation to  $x$  by finding the best possible combination of vectors from this Krylov subspace at each step to reduce the residual as much as possible

We know that  $\mathbf{b}$  is a linear combination of  $k$  eigenvectors of  $A$  and it has subspace  $\mathcal{K}_k(A, \mathbf{b})$ . This subspace contains the exact solution after  $k$  iterations. This is because after at most  $k$  matrix-vector multiplication, GMRES will have explored the entire span of the eigenvectors contributing to  $\mathbf{b}$ , thus leading to the exact solution.

The GMRES method can be interpreted as finding a polynomial  $p_i(A)$  at each iteration  $i$  such that  $r_i = p_i(A)r_0$  and this polynomial  $p_i(A)$  is chosen to minimize the residual. Since  $\mathbf{b}$  is a linear combination of  $k$  eigenvectors, the residual becomes zero when GMRES finds a polynomial that completely 'destroys' the contribution from all of those eigenvectors.

That is, more formally, after  $k$  iterations, GMRES will have found a polynomial  $p_k(A)$  of degree  $k-1$  such that  $p_k(A)r_0 = 0$ . This means that the residual is exactly zero after at most  $k$  iterations. This happens because the polynomial can be expressed as a linear combination of the eigenvalues of  $A$ , and once 'used up' all  $k$  eigenvectors, there is no further residual left to minimize.

Hence, the structure of  $b$  guarantees that GMRES will explore all necessary directions in at most  $k$  iterations. Once the Krylov subspace contains all the eigenvectors that contribute to  $\mathbf{b}$ , GMRES can find the exact solution, and no further iterations are needed.

Thus, GMRES terminates in at most  $k$  iterations, corresponding to the number of distinct eigenvectors involved in

Convergence curve:

When  $\mathbf{b}$  is a linear combination of  $k$  eigenvectors, GMRES will show rapid convergence. The residual norm  $\|r_i\|$  will decrease significantly at each iteration and hence, so will  $\frac{\|r_i\|}{\|b\|}$ . After exactly  $k$  iterations, the residual will be zero. The curve would thus be steep, showing termination after  $k$  iterations. (Similar to the one shown in Fig.35.3 in the book [Lloyd N. Trefethen and David Bau III, *Numerical Linear Algebra*])

## 22. Task:

In this task, we implement the GMRES method and test it on the linear system

$$Ax = b, \tag{121}$$

where

$$A = \frac{1}{h} \begin{bmatrix} 1 & & & -1 & 1 & & \ddots & \ddots \\ & -1 & 1 & & & & & \\ & & & & & & & \end{bmatrix} \in \mathbb{R}^{m \times m}, \quad h = \frac{1}{m-1}. \tag{122}$$

The right hand side vector  $b$  consists of 2's in the first half of the vector and 1's in the second half.

We test the method for  $m = 100$  and  $m = 1000$ , using a fixed number of 100 GMRES iterations. We also use the scipy implementation of the ILU preconditioner and compare the residuals in the 2-norm.

GMRES stands for Generalized Minimal Residual method. It is an iterative method for solving a linear system of the form given in equation 121.

Starting with an initial guess  $x_0$ , the initial residual is

$$r_0 = b - Ax_0. \quad (123)$$

The GMRES method searches for an approximate solution in the Krylov subspace

$$\mathcal{K}_k(A, r_0)$$

$$\text{span } r_0, Ar_0, A^2r_0, \dots, A^{k-1}r_0. \quad (124)$$

At the  $k$ -th iteration, the approximate solution has the form

$$x_k = x_0 + Q_k y_k, \quad (125)$$

where the columns of  $Q_k$  form an orthonormal basis of the Krylov subspace in equation 22.

The Arnoldi iteration gives the relation

$$AQ_k = Q_{k+1} \overline{H}_k, \quad (126)$$

where  $\overline{H}_k$  is an upper Hessenberg matrix.

GMRES chooses  $y_k$  such that the residual norm is minimized. That is,

$$y_k$$

$$\arg \min_y |\beta e_1 - \overline{H}_k y|_2, \quad \beta = |r_0|_2. \quad (127)$$

Then the new approximation is computed from equation 125.

For the preconditioned case, we use an ILU preconditioner. If  $P$  is the ILU approximation of  $A$ , then instead of solving equation 121 directly, we solve the left-preconditioned system

$$P^{-1}Ax = P^{-1}b. \quad (128)$$

The purpose of the preconditioner is to make the system easier for GMRES to solve. Ideally,  $P^{-1}A$  is closer to the identity matrix than  $A$  itself. This usually reduces the number of iterations needed for convergence.

In the code below, we plot the residual norm

$$|b - Ax_k|_2 \quad (129)$$

over the iterations for the following four cases:

- $m = 100$ , without ILU preconditioning,
- $m = 100$ , with ILU preconditioning,
- $m = 1000$ , without ILU preconditioning,

- $m = 1000$ , with ILU preconditioning.

### Python Code:

```
import numpy as np
import matplotlib.pyplot as plt
import scipy.sparse as sp
import scipy.sparse.linalg as spla

def create_matrix_and_rhs(m):
    """
    Creates the matrix A and the right hand side vector b.

    """
    A is the lower bidiagonal matrix with 1 on the diagonal
    and -1 on the subdiagonal, multiplied by 1/h.
    """

    h = 1 / (m - 1)

    diagonal = np.ones(m)
    subdiagonal = -np.ones(m - 1)

    A = (1 / h) * sp.diags(
        diagonals=[subdiagonal, diagonal],
        offsets=[-1, 0],
        shape=(m, m),
        format="csc"
    )

    b = np.ones(m)
    b[:m // 2] = 2

    return A, b

def gmres(A, b, max_iterations=100, M_solve=None):
    """
    Implements the GMRES method.

    """
    If M_solve is not None, then left preconditioning is used.
    This means that we apply GMRES to

         $M^{-1} A x = M^{-1} b.$ 

    The function returns the approximate solution x and the
    true residuals  $\|b - A x_k\|_2$ .
    """

    m = len(b)
    x0 = np.zeros(m)

    def apply_operator(v):
        Av = A @ v

        if M_solve is None:
            return Av
        else:
            M_inv = spla.linalg.pinv(M)
            return M_inv @ Av
```



```

        return M_solve(Av)

# Initial residual
r0 = b - A @ x0

if M_solve is None:
    r0_tilde = r0
else:
    r0_tilde = M_solve(r0)

beta = np.linalg.norm(r0_tilde, 2)

# Store the true residual ||b - A x_k||_2
residuals = [np.linalg.norm(r0, 2)]

if beta == 0:
    return x0, residuals

# Arnoldi basis vectors
Q = np.zeros((m, max_iterations + 1))
Q[:, 0] = r0_tilde / beta

# Upper Hessenberg matrix
H = np.zeros((max_iterations + 1, max_iterations))

x = x0.copy()

for k in range(max_iterations):
    # Arnoldi step
    v = apply_operator(Q[:, k])

    for j in range(k + 1):
        H[j, k] = np.dot(Q[:, j], v)
        v = v - H[j, k] * Q[:, j]

    H[k + 1, k] = np.linalg.norm(v, 2)

    if H[k + 1, k] != 0 and k + 1 < max_iterations + 1:
        Q[:, k + 1] = v / H[k + 1, k]

    # Solve the least squares problem
    e1 = np.zeros(k + 2)
    e1[0] = beta

    y, _, _, _ = np.linalg.lstsq(
        H[:k + 2, :k + 1],
        e1,
        rcond=None
    )

    # Compute new approximation
    x = x0 + Q[:, :k + 1] @ y

    # Compute true residual
    true_residual = np.linalg.norm(b - A @ x, 2)
    residuals.append(true_residual)

return x, residuals
'''

```

```

# Number of GMRES iterations
max_iterations = 100

# Store all residual histories
results = {}

for m in [100, 1000]:
    A, b = create_matrix_and_rhs(m)
    '''
    # Case 1: GMRES without preconditioner
    x_gmres, residuals_gmres = gmres(
        A,
        b,
        max_iterations=max_iterations,
        M_solve=None
    )

    results[(m, "GMRES")] = residuals_gmres

    # Case 2: GMRES with ILU preconditioner
    ilu = spla.spilu(A)

    def M_solve(v):
        return ilu.solve(v)

    x_ilu, residuals_ilu = gmres(
        A,
        b,
        max_iterations=max_iterations,
        M_solve=M_solve
    )

    results[(m, "GMRES with ILU")] = residuals_ilu
    '''

# Plotting the residual norms
plt.figure(figsize=(10, 6))

for key, residuals in results.items():
    m, method = key
    iterations = range(len(residuals))
    plt.semilogy(iterations, residuals, label=f"m = {m}, {method}")

plt.xlabel("Iteration number")
plt.ylabel("Residual norm in 2-norm")
plt.title("GMRES residuals with and without ILU preconditioning")
plt.legend()
plt.grid(True)
plt.show()

```

From the plot, we compare the residual norms for the four cases.

For the cases without ILU preconditioning, GMRES reduces the residual gradually over the iterations. The convergence becomes more difficult when the dimension is increased from  $m = 100$  to  $m = 1000$ .

For the cases with ILU preconditioning, the residual decreases much faster. This is because the ILU preconditioner gives an approximation of the matrix  $A$ , and therefore  $P^{-1}A$  is easier for GMRES to solve than  $A$  itself.

In this example, the matrix  $A$  is sparse and lower bidiagonal, so it is important to store it in sparse matrix format. This is why the code uses `scipy.sparse.diags` to construct  $A$  and `scipy.sparse.linalg.spilu` to compute the ILU preconditioner.

Thus, the experiment shows that ILU preconditioning improves the convergence of GMRES, especially when the dimension of the linear system becomes larger.

### 23. Task:

Let

$$A = \begin{pmatrix} 5 & 0 & 0 & -1 \\ 1 & 0 & -1 & 1 \\ -\frac{3}{2} & 1 & -2 & 1 \\ -1 & 1 & 3 & -3 \end{pmatrix}. \quad (130)$$

The matrix  $A(p)$  has the same diagonal entries as  $A$ , while all off-diagonal entries are multiplied by  $p$ . Hence

$$A(p) = \begin{pmatrix} 5 & 0 & 0 & -p \\ p & 0 & -p & p \\ -\frac{3}{2}p & p & -2 & p \\ -p & p & 3p & -3 \end{pmatrix}, \quad 0 \leq p \leq 1. \quad (131)$$

For  $p = 0$ , the matrix  $A(0)$  is diagonal:

$$A(0) = \begin{pmatrix} 5 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & -2 & 0 \\ 0 & 0 & 0 & -3 \end{pmatrix}. \quad (132)$$

Therefore the eigenvalues of  $A(0)$  are exactly the diagonal entries:

$$\lambda(A(0)) = \{5, 0, -2, -3\}. \quad (133)$$

Using `eig` numerically for  $p = 1$ , that is, for the original matrix  $A$ , we get

$$\lambda(A) \approx \{5.17335360, 0.35459639, -1.37985824, -4.14809175\}. \quad (134)$$

The characteristic polynomial of  $A(p)$ , computed as  $\det(\lambda I - A(p))$ , is

$$\det(\lambda I - A(p)) = \lambda^4 - (19 + 4p^2)\lambda^2 + \left(-\frac{11}{2}p^3 + 14p^2 - 30\right)\lambda + \frac{7}{2}p^4 + 12p^3 - 5p^2. \quad (135)$$

For each value of  $p \in [0, 1]$ , the eigenvalues are the roots of the polynomial in (135). In the code below, the eigenvalues are computed directly using `eig`.

For Gershgorin's theorem, the Gershgorin discs of  $A$  are centered at the diagonal entries  $a_{ii}$ , with radii

$$r_i = \sum_{j \neq i} |a_{ij}|. \quad (136)$$

Thus, for the matrix  $A$ , the four discs are

$$\begin{aligned} D_1 &= \{z \in \mathbb{C} : |z - 5| \leq 1\}, \\ D_2 &= \{z \in \mathbb{C} : |z| \leq 3\}, \\ D_3 &= \{z \in \mathbb{C} : |z + 2| \leq \frac{7}{2}\}, \\ D_4 &= \{z \in \mathbb{C} : |z + 3| \leq 5\}. \end{aligned} \tag{137}$$

For the matrix  $A(p)$ , the centers are the same, but the radii are multiplied by  $p$ . Hence the Gershgorin discs shrink to the points  $5, 0, -2, -3$  when  $p = 0$ , and become the discs in (137) when  $p = 1$ .

Gershgorin's theorem says that every eigenvalue of a matrix lies in the union of its Gershgorin discs. Therefore, every eigenvalue of  $A$  must lie in

$$D_1 \cup D_2 \cup D_3 \cup D_4. \tag{138}$$

This is exactly what the plot shows: the eigenvalue curves start at the diagonal entries of  $A$ , move as  $p$  increases from 0 to 1, and the final eigenvalues of  $A$  lie inside the union of the Gershgorin discs.

```
import numpy as np
import matplotlib.pyplot as plt
from itertools import permutations

# Matrix A
A = np.array([
    [5, 0, 0, -1],
    [1, 0, -1, 1],
    [-1.5, 1, -2, 1],
    [-1, 1, 3, -3]
], dtype=float)

# Diagonal part and off-diagonal part
D = np.diag(np.diag(A))
E = A - D

def A_of_p(p):
    """
    Construct A(p), where diagonal entries are fixed
    and off-diagonal entries are multiplied by p.
    """
    return D + p * E

# Values of p
ps = np.linspace(0, 1, 401)

# Store eigenvalues
eig_paths = np.zeros((len(ps), 4), dtype=complex)

# At p = 0 the eigenvalues are exactly the diagonal entries
eig_paths[0, :] = np.diag(A)

# Track the eigenvalues continuously as p changes.
# The ordering returned by eig is arbitrary, so at every step
# we match the new eigenvalues to the previous ones by closest distance.
for k, p in enumerate(ps[1:], start=1):
    w, V = np.linalg.eig(A_of_p(p))

    previous = eig_paths[k - 1, :]
```

```

    best_perm = min(
        permutations(range(4)),
        key=lambda perm: np.sum(np.abs(w[list(perm)] - previous))
    )

    eig_paths[k, :] = w[list(best_perm)]

# Eigenvalues of A = A(1)
wA, VA = np.linalg.eig(A)

print("Eigenvalues of A = A(1):")
for val in np.sort_complex(wA):
    print(val)

# Gershgorin disc centers and radii for A
centers = np.diag(A)
radii = np.sum(np.abs(A), axis=1) - np.abs(np.diag(A))

# Plot
fig, ax = plt.subplots(figsize=(8, 6))

# Plot eigenvalue paths
for j in range(4):
    ax.plot(
        eig_paths[:, j].real,
        eig_paths[:, j].imag,
        linewidth=2,
        label=f"eigenvalue path {j+1}"
    )

# Mark diagonal entries of A
ax.scatter(
    centers,
    np.zeros_like(centers),
    marker="x",
    s=100,
    linewidths=2,
    color="black",
    label="diagonal entries of A"
)

# Mark eigenvalues of A with fat circles
ax.scatter(
    wA.real,
    wA.imag,
    facecolors="none",
    edgecolors="black",
    s=250,
    linewidths=3,
    label="eigenvalues of A"
)

# Plot Gershgorin discs for A
theta = np.linspace(0, 2*np.pi, 500)

for i, (c, r) in enumerate(zip(centers, radii), start=1):
    x = c + r*np.cos(theta)
    y = r*np.sin(theta)

```

```

    ax.plot(x, y, linestyle="--", linewidth=1.5)
    ax.text(c, r + 0.2, f"D{i}", ha="center")

# Axes and formatting
ax.axhline(0, linewidth=0.8, color="gray")
ax.axvline(0, linewidth=0.8, color="gray")

ax.set_xlabel("Real part")
ax.set_ylabel("Imaginary part")
ax.set_title("Eigenvalues of A(p) and Gershgorin discs")
ax.set_aspect("equal", adjustable="box")
ax.grid(True)
ax.legend(loc="best")

plt.savefig("task1_eigenvalue_paths.pdf", bbox_inches="tight")
plt.show()

```